Chapter 19 — Interview Question Bank

How to use this bank

This chapter collects the JavaScript questions interviewers ask most, worded the way they usually appear in LeetCode study lists (above all **30 Days of JavaScript**) and the big public question collections listed at the end. It covers the language (ES2025), the browser, Node 24 and basic TypeScript.

- ⋆ marks a question that shows up in many sources: expect it.
- Each question has a **short answer you can say out loud** in about 30 seconds, or a short code snippet plus a sentence or two. Answers are written in this book's own words.
- → **More** links to the section of this book that explains the topic in full. Chapter 17 already answers 53 tricky questions with real output, and §17.3 holds tested implementations of the coding classics; this bank points there instead of repeating them, and adds every common question that chapter lacks.
- Code answers are correct for Node 24 and current browsers. Outputs in the puzzle group were checked by running them.

How to drill. Cover the answer, say yours out loud (or type the code), then uncover and compare. Mark the ones you missed and come back to them the next day. Do the ⋆ questions first, then the coding tasks (groups 14 and 15) from memory, with a timer.

Terms used everywhere below. A **primitive** is a simple immutable value (a string, a number…). An **object** is a collection of properties held by **reference** (variables store a pointer to it, not a copy). A **callback** is a function you pass to another function so it can call it later. A **polyfill** is your own code that adds a built-in feature an engine lacks. The **event loop** is the mechanism that runs queued work once the current code finishes.

---

1. Types, values and coercion

Q1. ⋆ What are the data types in JavaScript? There are eight. Seven are **primitives**: `string`, `number` (64-bit floating point), `bigint` (whole numbers of any size), `boolean`, `undefined`, `null` and `symbol` (a unique identifier). The eighth is `object`, which includes arrays, functions, dates, maps and everything else built from objects. Primitives are immutable and compared by value; objects are mutable and compared by reference. → *More:* §2.3 Data types and `typeof`

Q2. ⋆ What is the difference between primitive and reference (non-primitive) values? A primitive is stored directly in the variable; copying the variable copies the value, and nothing can change it in place. An object lives on the heap (the memory area for dynamically created data) and the variable holds a reference to it; copying the variable copies the reference, so both names see the same object. That is why `{} === {}` is `false` (two different objects) but `a === a` is `true`. → *More:* §2.4 Primitives vs objects

Q3. ⋆ What is the difference between `==` and `===`? `===` (strict equality) is `true` only if both values have the same type and value, with no conversion. `==` (loose equality) first **coerces** (automatically converts) the operands to a common type using a set of rules: `null == undefined` is `true`, strings and booleans become numbers, objects become primitives. Use `===` always; the one idiom some teams allow is `x == null` to test for "null or undefined". → *More:* §2.6 Comparison operators and equality · §17.4.1 Q2–Q3

Q4. ⋆ What is the difference between `null`, `undefined` and an undeclared variable? How do you check each? `undefined` means "no value was ever assigned" (a declared variable, a missing property, a missing argument). `null` is a value a programmer assigns on purpose to mean "empty". An **undeclared** variable was never declared at all: reading it throws `ReferenceError`. Check with `x === undefined`, `x === null`, `x == null` (either), and `typeof x === 'undefined'`, which is the only test that does not throw for an undeclared name. → *More:* §2.3 Data types and `typeof`

Q5. ⋆ What does `typeof` return, and why is `typeof null` `"object"`? `typeof` returns one of `"string"`, `"number"`, `"bigint"`, `"boolean"`, `"undefined"`, `"symbol"`, `"object"` or `"function"`. Arrays give `"object"` (use `Array.isArray`), classes give `"function"`. `typeof null === "object"` is a mistake in the original 1995 implementation: values carried a type tag, the tag for objects was 0, and `null` (a zero pointer) was read as an object. It stays because changing it now would break existing websites. → *More:* §17.4.1 Q4 and Q7

Q6. ⋆ What is type coercion? What are its common pitfalls? Coercion is JavaScript converting a value to another type automatically. `+` with any string concatenates (`1 + '2'` is `'12'`), while `-`, `*`, `/` convert to numbers (`'5' - 2` is `3`). `if`, `!`, `&&`, `||` convert to boolean. Pitfalls: `==` surprises, `'' == 0` is `true`, `[] + {}` is `'[object Object]'`, form inputs are strings so `input.value + 1` concatenates. Convert explicitly with `Number()`, `String()`, `Boolean()`. → *More:* §2.12 Type conversion summary · §17.4.1 Q1

Q7. ⋆ Which values are falsy? Exactly eight: `false`, `0`, `-0`, `0n`, `''` (empty string), `null`, `undefined` and `NaN`. Everything else is truthy, including `'0'`, `'false'`, `[]`, `{}` and `new Boolean(false)` (an object). → *More:* §2.11 Booleans, truthy and falsy

Q8. ⋆ What is `NaN`? What is the difference between `isNaN` and `Number.isNaN`? `NaN` ("not a number") is the number value produced by a failed numeric operation, such as `Number('abc')` or `0 / 0`. It is the only value not equal to itself (`NaN === NaN` is `false`). The global `isNaN(x)` converts `x` to a number first, so `isNaN('abc')` is `true`; `Number.isNaN(x)` is `true` only for the actual `NaN` value. Prefer `Number.isNaN` or `Object.is(x, NaN)`. → *More:* §2.10 Numbers in depth · §17.4.1 Q5

Q9. What is the difference between `Object.is` and `===`? They agree except in two cases: `Object.is(NaN, NaN)` is `true` (but `NaN === NaN` is `false`), and `Object.is(0, -0)` is `false` (but `0 === -0` is `true`). React uses `Object.is` to decide whether state changed. → *More:* §17.4.1 Q6

Q10. ⋆ Why is `0.1 + 0.2 !== 0.3`? How do you compare decimals? Numbers are IEEE 754 binary floating point, and 0.1 and 0.2 have no exact binary form, so the sum is `0.30000000000000004`. Compare with a tolerance: `Math.abs(a - b) < Number.EPSILON` (or a tolerance that fits the scale), round for display with `toFixed`, and store money as integer cents (or `BigInt`). → *More:* §2.10 Numbers in depth · §17.4.1 Q14

Q11. What is `Number.MAX_SAFE_INTEGER`, and what is `BigInt`? `Number.MAX_SAFE_INTEGER` is $2^{53} - 1$ (9,007,199,254,740,991): the largest integer a `number` can hold exactly, after which `n + 1` may equal `n + 2`. `BigInt` (`123n`, `BigInt('123')`) stores integers of any size. You cannot mix `BigInt` and `number` in arithmetic (`1n + 1` throws `TypeError`), `BigInt` has no decimals, and `JSON.stringify` cannot serialize it without a custom replacer. → *More:* §2.10 Numbers in depth · §17.1

Q12. ⋆ What is the difference between `||` and `??`? What does `?.` do? `a || b` returns `b` when `a` is any falsy value, so `0 || 10` is `10`. `a ?? b` (nullish coalescing) returns `b` only when `a` is `null` or `undefined`, so `0 ?? 10` is `0` — right for defaults where `0` or `''` are valid. `a?.b` (optional chaining) returns `undefined` instead of throwing when `a` is `null`/`undefined`; it also works as `a?.[key]` and `fn?.()`. → *More:* §2.7 Logical operators, `??` and `?.` · §17.4.1 Q12

Q13. What do `"hello" || "world"` and `"foo" && "bar"` return? `"hello"` and `"bar"`. `||` and `&&` return one of their **operands**, not a boolean: `||` returns the first truthy operand (or the last one), `&&` returns the first falsy operand (or the last one). This is called short-circuit evaluation: the right side is not evaluated if the left side decides the result. → *More:* §2.7

Q14. What does `!!` do? It converts any value to its boolean: the first `!` converts and negates, the second negates back. `!!'text'` is `true`, `!!0` is `false`. `Boolean(x)` does the same and reads more clearly.

Q15. What are logical assignment operators? ES2021 added `a ||= b` (assign if `a` is falsy), `a &&= b` (assign if `a` is truthy) and `a ??= b` (assign if `a` is `null`/`undefined`). They short-circuit: if no assignment is needed, `b` is not evaluated and no write happens. Common use: `options.timeout ??= 5000`. → *More:* §2.8 Bitwise and assignment operators

Q16. How do you convert a string to a number? `Number` vs `parseInt` vs unary `+`? `Number(s)` and `+s` convert the whole string and give `NaN` if any part is not numeric (`Number('12px')` is `NaN`); empty or whitespace-only strings give `0`. `parseInt(s, 10)` reads digits from the start and stops at the first non-digit (`parseInt('12px', 10)` is `12`); always pass the radix (the base, 10). `parseFloat` does the same for decimals. → *More:* §2.12 Type conversion summary

Q17. How is an object converted to a primitive? When an object is used where a primitive is needed (`+`, `==`, template literals), the engine calls `obj[Symbol.toPrimitive](hint)` if defined; otherwise, for the `"number"` or `"default"` hint, it tries `valueOf()` then `toString()`, and for the `"string"` hint `toString()` first. Plain objects end up as `'[object Object]'`, arrays as their elements joined by commas (`[1, 2] + ''` is `'1,2'`), dates as a readable string with `+` but a timestamp with `-`.

```
const price = { [Symbol.toPrimitive]: (hint) => (hint === 'number' ? 42 : '$42') };
console.log(+price, `${price}`); // 42 '$42'
```

Q18. What are `Symbol`s used for? A symbol (`Symbol('id')`) is a unique, immutable primitive used as a property key that can never clash with another key, even one with the same description. Uses: hidden-ish metadata on objects you don't own, and **well-known symbols** that hook into language behaviour — `Symbol.iterator` (make an object iterable), `Symbol.toPrimitive`, `Symbol.asyncIterator`, `Symbol.dispose`. Symbol keys are skipped by `for…in`, `Object.keys` and `JSON.stringify`. `Symbol.for('k')` gives a shared, registry-wide symbol. → *More:* §2.3 · §8.4 Iterators

Q19. How can `'abc'.toUpperCase()` work if strings are primitives? When you access a property on a primitive, the engine temporarily wraps it in its **wrapper object** (`String`, `Number`, `Boolean`, `Symbol`, `BigInt`), calls the method, and discards the wrapper. That's also why `str.foo = 1` silently does nothing (in sloppy mode) or throws in strict mode. Never create wrappers yourself with `new String('x')`: they are objects, so they are truthy and `===` fails. → *More:* §6.4

Q20. How do you check whether a value is an array, a plain object, a number? `Array.isArray(v)` (works across iframes, unlike `instanceof Array`). Number: `typeof v === 'number' && Number.isFinite(v)`. Plain object: `v !== null && typeof v === 'object' && Object.getPrototypeOf(v) === Object.prototype` (or `=== null`). For a general type name, `Object.prototype.toString.call(v)` gives `'[object Date]'`, `'[object Map]'`, and so on. → *More:* §9.7 `instanceof` and type checks

---

2. Variables, hoisting, scope and closures

Q21. ⋆ What are the differences between `var`, `let` and `const`? `var` is **function-scoped** (visible in the whole function), can be re-declared, is hoisted and initialised to `undefined`, and at the top level creates a property on `window`. `let` and `const` are **block-scoped** (visible only inside the `{}` where they are declared), cannot be re-declared in the same scope, and sit in the temporal dead zone until their line runs. `const` must be initialised and cannot be reassigned — but the object it points to can still be changed. Default to `const`, use `let` when you reassign, avoid `var`. → *More:* §2.1 · §17.4.2 Q15

Q22. ⋆ What is hoisting? Before code in a scope runs, the engine registers all its declarations. That makes it look as if declarations were "hoisted" (moved) to the top. Function declarations are hoisted with their body, so you can call them before their line. `var` is hoisted and set to `undefined`. `let`, `const` and `class` are hoisted but left uninitialised (the TDZ), so reading them early throws. Function expressions and arrows follow the rule of the variable that holds them. → *More:* §2.2 · §4.6 Hoisting — the complete picture

Q23. ⋆ What is the temporal dead zone (TDZ)? The TDZ is the time between entering a scope and reaching a `let`/`const`/`class` declaration. During it the variable exists (it shadows any outer variable of the same name) but any access throws `ReferenceError: Cannot access 'x' before initialization`. Even `typeof x` throws inside the TDZ. It exists to catch use-before-define bugs.

```
let x = 'outer';
{ console.log(x); let x = 'inner'; } // ReferenceError, not 'outer'
```

→ *More:* §2.2 · §17.4.2 Q16

Q24. ⋆ What is the difference between `function foo() {}` and `const foo = function () {}`? The first is a **function declaration**: hoisted with its body, so it can be called anywhere in its scope. The second is a **function expression** assigned to a variable: it exists only after that line runs (before it, `foo` is in the TDZ for `const`/`let`, or `undefined` for `var`, giving `TypeError: foo is not a function`). Expressions can be anonymous or named; a name on an expression is visible only inside it (handy for recursion and stack traces). → *More:* §4.2 Declarations vs expressions

Q25. What is scope? What kinds are there? Scope is the region of code where a name is visible. JavaScript has **global** scope, **module** scope (top level of an ES module — not global), **function** scope, and **block** scope (any `{}` for `let`/`const`/`class`). When a name is used, the engine looks in the current scope, then the enclosing one, and so on up to global: this chain is the **scope chain**. If it is not found, reading throws `ReferenceError`. → *More:* §4.5 Scope

Q26. ⋆ What is lexical scope? Lexical (static) scope means a function's accessible variables are decided by **where it is written** in the source, not by where it is called. A function defined inside another can see the outer function's variables, wherever it is later called from. Closures are a consequence of lexical scope. (`this` is the exception: for regular functions it depends on the call.) → *More:* §4.5

Q27. What are shadowing and illegal shadowing? Shadowing is declaring a variable in an inner scope with the same name as an outer one; inside, the inner one hides the outer. **Illegal shadowing** is shadowing a `let`/`const` with a `var` in an inner block of the same function: because `var` belongs to the whole function, it would clash with the `let`, so it is a `SyntaxError`. Shadowing a `var` with a `let` in a block is fine.

```
function f() { let a = 1; { var a = 2; } } // SyntaxError: 'a' has already been declared
```

Q28. ⋆ What is a closure? Give a practical use. A closure is a function together with the variables of the scope it was created in. The function keeps access to those variables even after the outer function has returned, and each call of the outer function creates a fresh set. Uses: private state (counters, caches), function factories, `debounce`/`memoize`, event handlers that remember data, the module pattern.

```
function makeCounter() { let count = 0; return () => ++count; }
const next = makeCounter(); next(); next(); // 2 — `count` lives on in the closure
```

→ *More:* §4.7 Closures · §17.4.2 Q19

Q29. ⋆ What does this print, and how do you fix it? `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));` It prints `3 3 3`. `var` creates a single `i` for the whole function; the callbacks run after the loop, when `i` is already 3. Fix with `let i` (a `for` loop creates a fresh binding per iteration, so it prints `0 1 2`), or capture the value with an IIFE or `setTimeout(console.log, 0, i)`. → *More:* §17.4.2 Q18

Q30. How do you create private variables in JavaScript? Three ways: a **closure** (variables of a factory function, reachable only through the functions it returns); `#private` **class fields** (`#balance`, enforced by the language, ES2022); or a **module** (anything not exported is private to the file). A `WeakMap` keyed by instance is an older pattern. A leading underscore (`_x`) is only a naming convention. → *More:* §4.7 · §17.4.4 Q33

Q31. ⋆ What is an IIFE and why was it used? An IIFE (immediately invoked function expression) is a function that runs as soon as it is defined: `(function () { … })();` or `(() => { … })();`. Before `let`/`const` and modules, it was the way to create a private scope so variables didn't leak into the global scope, and to capture loop values. Today modules and block scope cover most uses; it's still handy for an `async` IIFE in old CommonJS code. → *More:* §4.9 IIFE

Q32. What is the module pattern? An IIFE that returns an object of public functions, while variables inside stay private through closure. It was the standard way to build modules before ES modules.

```
const bank = (() => { let balance = 0;
  return { deposit: (n) => (balance += n), get: () => balance }; })();
```

→ *More:* §4.9

Q33. ⋆ What is an execution context? What are the call stack and its phases? An execution context is the record the engine creates to run a piece of code: its variables (the **environment record**), the link to the outer scope, and the value of `this`. There is one global context, and a new one for each function call. Each has two phases: **creation** (declarations registered — this is hoisting — and `this` decided) and **execution** (code runs line by line). The **call stack** holds these contexts: a call pushes a context, a return pops it. A recursive function that never stops overflows it: `RangeError: Maximum call stack size exceeded`. → *More:* §4.6 · §12.1

Q34. ⋆ What is strict mode? How do you enable it? Strict mode is a stricter variant of JavaScript that turns silent mistakes into errors: assigning to an undeclared name throws instead of creating a global, `this` is `undefined` in a plain function call (not the global object), duplicate parameter names and `with` are forbidden, writing to read-only properties throws. Enable it with `'use strict';` at the top of a file or function. ES modules and class bodies are always strict. → *More:* §1.8 Strict mode · §17.4.2 Q22

Q35. What is `globalThis`? What problems do global variables cause? `globalThis` (ES2020) is the global object in every environment: `window` in a page, `self` in a worker, `global` in Node. Global variables are risky: any script can overwrite them, names collide between libraries, they stay in memory forever, and they make code hard to test. Keep state in modules, closures or passed-in parameters. → *More:* §4.5

Q36. What is wrong with `let a = b = 5;`? It is parsed as `let a = (b = 5)`. Only `a` is declared; `b` is assigned without a declaration, so in sloppy mode it becomes an accidental global variable, and in strict mode it throws `ReferenceError`. → *More:* §17.4.2 Q20

---

3. Functions and functional patterns

Q37. ⋆ How do arrow functions differ from regular functions? Arrow functions have no own `this`, `arguments`, `super` or `new.target`: they use the ones of the surrounding scope (lexical `this`). They cannot be called with `new` and have no `prototype`. They have a shorter syntax with implicit return for a single expression (wrap an object literal in parentheses: `() => ({ a: 1 })`). Use them for callbacks; use regular functions or methods for object methods that need `this`, and for constructors. → *More:* §4.3 Arrow functions · §17.4.3 Q24

Q38. What does "functions are first-class" mean? What is a higher-order function? First-class means functions are values: you can store them in variables, pass them as arguments, return them and put properties on them. A **higher-order function** takes a function as an argument and/or returns one: `map`, `filter`, `reduce`, `setTimeout`, `addEventListener`, and your own `debounce` or `memoize`. → *More:* §4.8 Higher-order functions and callbacks

Q39. What is a pure function? What are side effects? A pure function always returns the same output for the same input and has no side effects. A **side effect** is anything a function does besides returning a value: changing outside variables or its arguments, I/O (network, files, DOM, `console.log`), reading the clock or `Math.random()`. Pure functions are easy to test, cache (memoize) and run in parallel. **Referential transparency** means a call can be replaced by its result without changing the program — true for pure functions. → *More:* §4.11 Pure functions

Q40. What is a callback, and what is callback hell? A callback is a function passed to another function to be called later — when an event fires, a timer ends, or async work finishes. **Callback hell** is deeply nested callbacks for sequential async steps, each with its own error check, making code hard to read and error handling easy to forget. Promises and `async`/`await` flatten it. → *More:* §12.6 Callbacks and callback hell

Q41. ⋆ What is currying? Why is it useful? Currying turns a function of several arguments into a chain of functions taking one (or a few) arguments each: `add(1, 2, 3)` becomes `add(1)(2)(3)`. It lets you build specialised functions by fixing early arguments (`const tax = multiply(0.2)`), which suits `map` and composition. **Partial application** is related: fixing some arguments now (`fn.bind(null, a)`) and passing the rest later, without the one-at-a-time rule. → *More:* §17.3 Curry · Q255 below

Q42. What are `compose` and `pipe`? Both combine functions into one. `compose(f, g, h)(x)` is `f(g(h(x)))` — applies right to left, as in maths. `pipe(f, g, h)(x)` is `h(g(f(x)))` — left to right, the order you read. Implementation: `const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);` and `compose` uses `reduceRight`. → *More:* §17.3 Compose and pipe

Q43. ⋆ What is memoization? Memoization caches a function's results by its arguments so a repeat call returns the cached value instead of recomputing. It only makes sense for pure functions. The cache key is the tricky part (one primitive argument: use it directly; several: `JSON.stringify(args)` or a nested `Map`); unbounded caches leak memory, so cap them (LRU) or use a `WeakMap` for object keys. → *More:* §17.3 Memoize · Q257 below

Q44. ⋆ What is the difference between rest parameters and spread syntax? Both use `...`. **Rest** collects many values into one array: in a parameter list (`function f(...args)`) or a destructuring pattern (`const [first, ...others] = arr`). **Spread** expands one iterable or object into many: in a call (`Math.max(...nums)`), an array literal (`[...a, ...b]`) or an object literal (`{ ...defaults, ...opts }`). Rest must be last. → *More:* §4.4 Parameters

Q45. What is the `arguments` object, and how does it differ from rest parameters? `arguments` is an array-like object (indexed, with `length`, but no array methods) holding all arguments of a regular function call. Rest parameters give a real array, only of the arguments you didn't name, and also work in arrow functions (which have no `arguments` of their own). Prefer rest. → *More:* §4.4

Q46. How do default parameters work? When are they used? A default (`function f(x = 10)`) applies only when the argument is `undefined` (missing or passed as `undefined`); `null`, `0` and `''` do not trigger it. Defaults are evaluated at call time, left to right, and can refer to earlier parameters (`(a, b = a * 2)`). A parameter with a default (and everything after it) is not counted in `fn.length`. → *More:* §4.4

Q47. How do you find how many parameters a function expects? `fn.length` gives the number of parameters before the first one with a default or a rest parameter: `((a, b = 2, ...c) => 0).length` is `1`. The `curry` implementation uses it to know when enough arguments have arrived.

Q48. What is recursion? Does JavaScript have tail-call optimisation? Recursion is a function calling itself on a smaller input until a **base case** stops it. Each call uses a stack frame, so very deep recursion (around ten thousand frames, depending on the engine) throws `RangeError`. ES2015 specified **proper tail calls** (a call in return position reusing the frame), but only Safari's engine implements it; V8 (Chrome, Node) does not. For deep inputs, use a loop with an explicit stack. → *More:* §4.10 Recursion

Q49. Why should you avoid `eval` and `new Function`? Both run a string as code. `eval` runs it in the current scope, so it can read and change local variables; both are code-injection risks when the string contains user input, block engine optimisations, and are blocked by a strict Content Security Policy. Use `JSON.parse` for data and lookup tables or functions instead of building code from strings. → *More:* §14.12 CSP

Q50. What is a thunk? A thunk is a function that wraps a computation to delay it: `const thunk = () => expensive(a, b)`. Nothing runs until you call it. Async thunks take a callback (or return a promise) and are the idea behind lazy evaluation and the Redux `thunk` middleware (functions dispatched instead of plain action objects).

Q51. What is an anonymous function, and when do you use one? A function without a name, usually a function expression or arrow passed inline: `arr.map((x) => x * 2)`, `setTimeout(function () { … })`, an IIFE. Use it for short one-off callbacks. Name functions that are reused, recursive, or that you want readable in stack traces.

---

4. `this`, `call` / `apply` / `bind`, prototypes and classes

Q52. ⋆ How does `this` work in JavaScript? For regular functions, `this` is decided at **call time** by how the function is called, checked in this order: (1) `new Fn()` — the new object; (2) `fn.call/apply(obj)` or a function made by `bind(obj)` — `obj`; (3) `obj.method()` — `obj`, the object before the dot; (4) a plain call `fn()` — `undefined` in strict mode, the global object in sloppy mode. **Arrow functions** ignore all of this and use `this` from the scope where they were written. In event listeners (regular function), `this` is the element. → *More:* §9.1 `this` — the call-time context

Q53. ⋆ What is the difference between `call`, `apply` and `bind`? All three set `this` explicitly. `fn.call(obj, a, b)` calls immediately with arguments listed; `fn.apply(obj, [a, b])` calls immediately with arguments as an array; `fn.bind(obj, a)` does not call — it returns a new function with `this` (and optionally the first arguments) fixed permanently. Mnemonic: **a**pply = **a**rray; bind = later. → *More:* §9.2 · §17.4.3 Q26

Q54. ⋆ Why does a method lose `this` when passed as a callback? How do you fix it? `setTimeout(user.greet, 100)` passes only the function; when the timer calls it, it is a plain call, so `this` is `undefined` (or global). Fixes: `user.greet.bind(user)`, a wrapper arrow `() => user.greet()`, or define the method as an arrow class field (`greet = () => …`) so it captures the instance. → *More:* §17.4.3 Q23

Q55. What is `this` inside an arrow function defined as an object-literal property? It is **not** the object. An object literal does not create a scope, so the arrow takes `this` from the code around the literal — `undefined` at the top of a module, `window` in a classic script. Use method shorthand (`greet() { … }`) for methods. → *More:* §17.4.3 Q25 · Q226 below

Q56. What happens if you `bind` a function twice, or call a bound function with `new`? The first `bind` wins: `f.bind(a).bind(b)` still has `this === a` (the second bind can still add arguments). `call` / `apply` on a bound function cannot change its `this` either. But `new` beats `bind`: `new (Fn.bind(obj))()` creates a fresh object as `this`, ignoring `obj`. → *More:* §17.4.3 Q27

Q57. ⋆ Explain prototypal inheritance and the prototype chain. Every object has a hidden link, `[[Prototype]]`, to another object (or `null`). When you read a property the object doesn't have, the engine follows that link and looks there, then that object's prototype, and so on until `null`. That chain is how objects share methods: arrays get `map` from `Array.prototype`, which gets `hasOwnProperty` from `Object.prototype`. Writing a property always creates it on the object itself (shadowing the prototype's). Classes are built on exactly this mechanism. → *More:* §9.3 Prototypes and the prototype chain

Q58. ⋆ What is the difference between `prototype` and `__proto__`? `Fn.prototype` is a property of **functions** (constructors and classes): the object that becomes the prototype of instances created with `new Fn()`. `obj.__proto__` is the old accessor for an **object's own** prototype link; use `Object.getPrototypeOf(obj)` / `Object.setPrototypeOf` instead. So `new Fn().__proto__ === Fn.prototype`. → *More:* §17.4.4 Q29

Q59. ⋆ What does the `new` keyword do? `new Fn(args)` (1) creates an empty object whose prototype is `Fn.prototype`; (2) calls `Fn` with `this` set to that object; (3) returns that object — unless `Fn` explicitly returns another **object**, which then replaces it (returning a primitive is ignored). → *More:* §9.4 Constructor functions and `new` · §17.4.3 Q28 · Q265 below

Q60. Explain `function Person(){}`, `const p = Person()` and `const p = new Person()`. The first declares a function (meant as a constructor, by the capital letter convention). `Person()` is a plain call: it returns `undefined` (no `return`), so `p` is `undefined`, and any `this.name = …` inside writes to the global object (sloppy) or throws (strict). `new Person()` creates and returns a new object linked to `Person.prototype`.

Q61. What are the differences between ES6 classes and ES5 constructor functions? Classes are mostly cleaner syntax over the same prototype mechanism, with real differences: a class must be called with `new` (calling it plainly throws), class bodies are always strict mode, class declarations are in the TDZ until their line (not usable early like function declarations), methods are non-enumerable, and classes add `extends` / `super`, `static` members and blocks, fields and `#private` members. Subclassing built-ins like `Array` and `Error` works properly only with classes. → *More:* §9.5 Classes · §17.4.4 Q31

Q62. How does inheritance work with `extends` and `super`? How was it done before ES6? `class Dog extends Animal` links `Dog.prototype` to `Animal.prototype` (instance methods) and `Dog` to `Animal` (static methods). In the subclass constructor you must call `super(args)` before using `this`, because the parent creates the object. `super.method()` calls the parent's version. Before ES6: `Dog.prototype = Object.create(Animal.prototype); Dog.prototype.constructor = Dog;` and call `Animal.call(this, args)` in the `Dog` constructor. → *More:* §9.6 Inheritance with `extends` and `super` · §17.4.4 Q34

Q63. What are static members, private fields and static blocks? `static` members belong to the class itself, not instances: utility functions (`Math`-style helpers), factory methods (`User.fromJSON`), shared counters or caches. `#field` and `#method()` are **private**: accessible only inside the class body, a `SyntaxError` elsewhere; `#field in obj` checks whether an object has it. A `static { … }` block runs once when the class is defined, for complex static initialisation. → *More:* §9.5 Classes

Q64. What are getters and setters? Accessor properties that run a function when read (`get fullName() { … }`) or assigned (`set fullName(v) { … }`), while looking like plain properties to callers. Uses: computed values, validation on assignment, lazy initialisation, keeping a stable API while changing internals. A getter without a setter makes the property read-only (assignment throws in strict mode). → *More:* §5.7 Getters and setters

Q65. ⋆ How does `instanceof` work? `obj instanceof C` walks `obj`'s prototype chain and returns `true` if it meets `C.prototype`. It can be fooled: changing prototypes changes the answer, it fails across realms (an array from an iframe is not `instanceof Array` of your window — use `Array.isArray`), and a class can

override it with `static [Symbol.hasInstance]`. Primitives always give `false`. For errors, `Error.isError(v)` (ES2026, in Node 24) is the cross-realm check. → *More:* §9.7 · §17.4.4 Q30 · Q266 below

Q66. What are the ways to create an object? An object literal `{}`; `new` with a constructor function or class; `Object.create(proto)` (choose the prototype, or `null` for a bare dictionary); a factory function returning a literal; `Object.fromEntries(pairs)`; `structuredClone` or spread of an existing object. Literals and classes cover almost all real code. → *More:* §5.1

Q67. What is the difference between `Object.create()` and `Object.assign()`? `Object.create(proto, descriptors?)` makes a **new empty object whose prototype is `proto`** — it sets up inheritance, copying nothing. `Object.assign(target, ...sources)` **copies own enumerable properties** from sources into an existing target (a shallow copy, running setters on the target) and returns the target. `{ ...a, ...b }` is the modern form of `Object.assign({}, a, b)`. → *More:* §5.6 · §17.4.4 Q32

Q68. What is a mixin? Composition vs inheritance? A mixin is a set of methods copied into a class (or applied as a function `Base => class extends Base {…}`), adding behaviour without a deep inheritance chain. **Composition** builds objects from smaller parts they *have* ("has-a") rather than inheriting what they *are* ("is-a"); it avoids fragile deep hierarchies and is generally preferred. Use inheritance for a true, shallow "is-a" relationship. → *More:* §9.8 Mixins · §9.9 Composition vs inheritance

Q69. What are the pros and cons of extending built-in prototypes (e.g. adding `Array.prototype.last`)? Pro: convenient syntax everywhere. Cons: it changes every array in the program, including in libraries; names can clash with future standard methods (this is why the standard had to call it `flat` instead of `flatten` — a library had already added a different `flatten`); and `for…in` can expose enumerable additions. Only acceptable for spec-compliant polyfills; otherwise write a helper function. → *More:* Q292 below (the LeetCode version)

Q70. How do you implement method chaining? Return `this` from each method that changes state, so the next call works on the same object: `calc.add(5).multiply(2).getResult()`. For immutable chaining (like array `map().filter()`), return a **`new`** instance instead. → *More:* Q300 below

Q71. How do you implement a singleton in JavaScript? The simplest singleton is an ES module that exports one instance: modules are evaluated once and cached, so every importer gets the same object. With a class, keep a `static #instance` and return it from a `static getInstance()` (or from the constructor). Singletons are global state in disguise, so they make tests harder; prefer passing dependencies in. → *More:* §10.3 How ES modules load

Q72. What are `Proxy` and `Reflect`? A `Proxy` wraps an object and intercepts operations on it through **traps** — `get`, `set`, `has`, `deleteProperty`, `apply`, `construct` and more. Uses: validation, default values, logging, change tracking for reactivity (Vue 3 uses proxies), negative array indexes. `Reflect` provides the default behaviour of each operation as a function (`Reflect.get(target, key, receiver)`), so a trap can do its extra work and then forward correctly.

```
const safe = new Proxy({}, { get: (t, k) => (k in t ? t[k] : `no ${String(k)}`) });
console.log(safe.name); // 'no name'
```

Q73. Discuss using object-oriented principles in JavaScript. JavaScript supports the four pillars: **encapsulation** (`#private` fields, closures, modules), **abstraction** (expose a small public API), **inheritance** (prototypes / `extends`), and **polymorphism** (any object with the right method works — "duck typing" — and subclasses override methods). In practice JavaScript code mixes OOP with functional style: classes for things with identity and lifecycle, plain functions and data elsewhere, composition over deep inheritance. → *More:* §9.9 · LLD Handbook

---

5. Objects, arrays, strings and collections

Q74. ⋆ What is the difference between a shallow copy and a deep copy? How do you deep-copy an object? A **shallow copy** (`{ ...obj }`, `Object.assign({}, obj)`, `[...arr]`, `arr.slice()`) copies only the top level; nested objects are still shared, so changing `copy.address.city` changes the original. A **deep copy** copies every level. Use `structuredClone(obj)` (built in; handles `Date`, `Map`, `Set`, typed arrays and circular references, but throws on functions and DOM nodes and drops prototypes/class identity).

`JSON.parse(JSON.stringify(obj))` is lossy: it drops `undefined` and functions, turns dates into strings, and throws on cycles. → *More:* §5.4 Spread, rest and copying · §17.3 Deep clone · §17.4.6 Q43

Q75. Explain mutable vs immutable. How do you make an object immutable? A mutable value can be changed in place (objects, arrays); an immutable one cannot (primitives — string methods always return new strings). Immutability makes change detection cheap (compare references, as React does) and prevents spooky action at a distance; the cost is extra copying. Tools: `const` (prevents reassignment only), `Object.freeze` (shallow — freeze recursively for deep), non-mutating methods (`toSorted`, `with`, spread), or a library like Immer. → *More:* §5.6 · §17.4.6 Q44

Q76. What is the difference between `Object.freeze`, `Object.seal` and `Object.preventExtensions`? And `const` vs `freeze`? `preventExtensions`: no new properties. `seal`: also no deleting, and existing properties become non-configurable — values can still change. `freeze`: also no changing values; everything is read-only. All three are **shallow** (nested objects stay mutable), and violations fail silently in sloppy mode but throw `TypeError` in strict mode. `const` is about the **variable** (cannot be reassigned); `freeze` is about the **object** (cannot be modified). They are independent. → *More:* §5.6

Q77. What are property descriptors? Each property has attributes: a data property has `value`, `writable`, `enumerable` (shows in `for…in` / `Object.keys`), `configurable` (can be deleted or redefined); an accessor property has `get` / `set` instead of `value` / `writable`. Read them with `Object.getOwnPropertyDescriptor(s)`, set them with `Object.defineProperty`. Properties created by `defineProperty` default to all `false`; normal assignment makes them all `true`. → *More:* §5.6

Q78. ⋆ How do you check if a key exists in an object? `Object.hasOwn(obj, 'k')` (ES2022) — own property only, the recommended check. `'k' in obj` — own **or inherited** (so `'toString' in {}` is `true`). `obj.k !== undefined` fails when the value is legitimately `undefined`. `obj.hasOwnProperty('k')` breaks for `Object.create(null)` objects or an object with its own `hasOwnProperty` key — hence `Object.hasOwn`. → *More:* §5.2

Q79. How do you loop over an object's properties? How do you check whether an object is empty? `for (const [k, v] of Object.entries(obj))` is the usual way; `Object.keys` and `Object.values` give keys or values. All three include only **own enumerable string** keys. `for…in` also visits inherited enumerable keys. Empty check: `Object.keys(obj).length === 0` (for a plain object). Order: integer-like keys first in ascending order, then other string keys in insertion order. → *More:* §5.6 · Q290 below

Q80. ⋆ What is the difference between `for…in` and `for…of`? `for…in` loops over the **enumerable property names** (strings) of an object, including inherited ones — meant for plain objects. `for…of` loops over the **values** of an **iterable** (arrays, strings, `Map`, `Set`, generators, `NodeList`) — use it for arrays. Using `for…in` on an array gives string indexes and any extra properties; `for…of` on a plain object throws `TypeError: obj is not iterable`. → *More:* §3.9 · §3.10 · §17.4.6 Q50

Q81. Give examples of object and array destructuring. Destructuring unpacks values into variables. You can rename, set defaults, nest, and collect the rest.

```
const { name, age: years = 0, address: { city } = {}, ...rest } = user;
const [first, , third = 'n/a', ...others] = list;
[a, b] = [b, a];                          // swap without a temp variable
function draw({ x = 0, y = 0 } = {}) {}     // named options with defaults
```

→ *More:* §5.5 · §7.7

Q82. ⋆ What are `JSON.stringify` and `JSON.parse`? What does `stringify` drop? `JSON.stringify(value, replacer?, indent?)` turns a value into a JSON string; `JSON.parse(text, reviver?)` turns JSON text back into values. `stringify` skips properties whose value is `undefined`, a function or a symbol (in arrays they become `null`), turns `NaN` / `Infinity` into `null`, calls `toJSON` (so dates become ISO strings), throws on `BigInt` and on circular references. A **replacer** filters or transforms values on the way out; a **reviver** transforms them on the way in (e.g. turning date strings back into `Date`). → *More:* §5.12 JSON · §17.4.6 Q49

Q83. How do you handle circular references when serializing? `JSON.stringify` throws `TypeError: Converting circular structure to JSON`. Options: use `structuredClone` (if you only need a copy), pass a replacer that tracks seen objects in a `WeakSet` and replaces repeats with a marker (simple, but it also marks an object that is merely referenced twice, not only true cycles), or restructure the data to use IDs instead of back-references.

```
const seen = new WeakSet();
JSON.stringify(obj, (k, v) => (typeof v === 'object' && v !== null ? (seen.has(v) ? '[Circular]' : (seen.add(v), v)) : v));
```

Q84. ⋆ When do you use a `Map` instead of an object? And `Set`? Use a `Map` for a dictionary with keys of any type (objects, numbers kept as numbers), frequent adds and deletes, a reliable `size`, guaranteed insertion order and no inherited keys (`'constructor'` can't clash). Use an object for fixed-shape records and JSON. A `Set` stores unique values (by `SameValueZero`, so `NaN` works) with O(1) `has`; ES2025 added set algebra: `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`. → *More:* §8.1 `Map` · §8.2 `Set` · §17.4.6 Q48

Q85. ⋆ What are `WeakMap` and `WeakSet`? How do they differ from `Map` and `Set`? Their keys must be objects (or non-registered symbols), and they hold them **weakly**: if nothing else references a key, it can be garbage-collected and its entry disappears. Therefore they are not iterable and have no `size` or `clear`. Use them to attach data to objects you don't own (DOM nodes, cache per object, private data) without causing memory leaks. → *More:* §8.3 · §17.4.7 Q53

Q86. ⋆ What is the difference between `map` and `forEach`? `map` returns a **new array** of the callback's return values (same length); use it to transform. `forEach` returns `undefined` and is only for side effects (logging, writing to the DOM). Neither can be stopped early with `break` (use `for…of`, `some` or `find`), and neither waits for `async` callbacks. → *More:* §7.5 Iteration methods

Q87. ⋆ What is the difference between `slice` and `splice`? `slice(start, end)` returns a **copy** of a portion (end exclusive), leaving the original untouched; negative indexes count from the end. `splice(start, deleteCount, ...items)` **mutates** the array — removing and/or inserting in place — and returns the removed elements. ES2023's `toSpliced` is the non-mutating version of `splice`. → *More:* §7.4 The methods table

Q88. Which array methods mutate the array? What are the non-mutating alternatives? Mutating: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `copyWithin` (and setting `length`). ES2023 added copies: `toSorted`, `toReversed`, `toSpliced`, and `with(i, v)` (copy with one element replaced). Non-mutating methods also include `map`, `filter`, `slice`, `concat`, `flat`. This matters for React state and for functions that receive arrays they don't own. → *More:* §7.4

Q89. Explain `reduce` with examples. `arr.reduce((acc, item, i, arr) => newAcc, initial)` walks the array carrying an **accumulator**; each step returns the next accumulator, and the last one is the result. Always pass `initial` (without it, the first element is used, and an empty array throws `TypeError`).

```
const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
const countBy = words.reduce((m, w) => ((m[w] = (m[w] ?? 0) + 1), m), {});
```

→ *More:* §7.5

Q90. ⋆ How do you sort numbers correctly? Is `sort` stable? Default `sort()` converts elements to strings and compares UTF-16 code units, so `[10, 1, 2].sort()` gives `[1, 10, 2]`. Pass a comparator: `(a, b) => a - b` ascending; for strings use `a.localeCompare(b)`; for objects compare a field. Since ES2019, `sort` is required to be **stable** (equal elements keep their order), so you can sort by a secondary key first, then a primary key. `sort` mutates; use `toSorted` for a copy. → *More:* §7.6 Sorting correctly · §17.4.1 Q11

Q91. ⋆ How do you remove duplicates from an array? Primitives: `[...new Set(arr)]` (or `Array.from(new Set(arr))`). Objects by a key: `[...new Map(arr.map((o) => [o.id, o])).values()]` keeps the **last** object per id (in the position where that id first appeared); to keep the **first**, filter with a `Set` of seen ids — `const seen = new Set(); arr.filter((o) => !seen.has(o.id) && seen.add(o.id))`. `arr.filter((x, i) => arr.indexOf(x) === i)` works but is O(n²). → *More:* §8.2

Q92. What are `find`, `findIndex`, `findLast`, `some`, `every`, `includes` and `indexOf` for? `find`/`findIndex` return the first matching element/index (`undefined`/`-1` if none); `findLast`/`findLastIndex` (ES2023) search from the end. `some` is `true` if any element matches, `every` if all do (an empty array gives `true` for `every`). `includes(x)` checks presence using SameValueZero, so it finds `NaN`; `indexOf` uses `===` and cannot. → *More:* §7.5

Q93. How do you empty an array? What do `delete arr[i]` and `arr.length = 0` do? `arr.length = 0` empties the array in place (every reference sees it empty). `arr = []` makes the variable point to a new array (others still see the old one; impossible with `const`). `arr.splice(0)` also empties in place and returns

the removed items. `delete arr[i]` leaves a **hole** (`length` unchanged); use `splice` to remove. → *More:* §17.4.6 Q46

Q94. What are sparse arrays? What is the difference between `Array(3)` and `Array.of(3)`? A sparse array has **holes** — indexes with no element at all (`[1, , 3]`, `Array(3)`, `delete arr[1]`, or writing past the end). `forEach` / `map` / `filter` skip holes; `for…of` and spread treat them as `undefined`. `Array(3)` makes an empty array of length 3; `Array(3, 4)` and `Array.of(3)` make arrays of those elements. To create filled arrays use `Array.from({ length: 3 }, (_, i) => i)` or `Array(3).fill(0)`. → *More:* §7.2 Creating arrays

Q95. What is an array-like object? How do you convert it to an array? An object with numeric keys and a `length`, but without array methods — `arguments`, a `NodeList` from older APIs, a `{ length: 2, 0: 'a', 1: 'b' }`. Convert with `Array.from(x)` (works for array-likes and iterables, and takes a map function) or `[...x]` (iterables only). → *More:* §7.2

Q96. What is an iterator? How do you make an object iterable? An **iterator** is an object with a `next()` method that returns `{ value, done }`. An **iterable** is an object with a `[Symbol.iterator]()` method returning an iterator; `for…of`, spread, destructuring, `Array.from`, `Promise.all` and `new Map()` all consume iterables. Easiest way to make one: a generator method.

```
class Range { constructor(a, b) { this.a = a; this.b = b; }
  *[Symbol.iterator]() { for (let i = this.a; i <= this.b; i++) yield i; } }
console.log([...new Range(1, 3)]); // [1, 2, 3]
```

→ *More:* §8.4

Q97. What are generators and when would you use them? A generator function (`function*`) returns an iterator; each `yield` pauses the function and hands out a value, and the next `next()` resumes it (and `next(x)` makes the paused `yield` evaluate to `x`). Uses: lazy sequences (including infinite ones), custom iteration, paging through results, and (historically) async control flow before `async` / `await`. **Async generators** (`async function*`) yield promises' results and are consumed with `for await…of`. → *More:* §8.5 Generators · §12.12

Q98. What are iterator helpers (ES2025)? Methods on iterators — `map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, `forEach`, `some`, `every`, `find` — plus `Iterator.from`. Unlike array methods, they are **lazy**: they process one element at a time and stop early, so they work on huge or infinite sequences without building intermediate arrays: `naturals().filter(isPrime).take(5).toArray()`. → *More:* §8.6 Iterator helpers (ES2025)

Q99. What do `Object.groupBy` and `Map.groupBy` do? ES2024. `Object.groupBy(items, fn)` returns a null-prototype object whose keys are the callback's return values and whose values are arrays of the items in each group; `Map.groupBy` returns a `Map`, so keys can be any value (objects, numbers). They replace the classic `reduce` grouping. → *More:* §8.7 · Q293 below

Q100. Are strings mutable? `slice` vs `substring` vs `substr`? Strings are immutable: every method returns a new string and `s[0] = 'x'` does nothing. `slice(start, end)` accepts negative indexes (from the end). `substring(start, end)` treats negatives as 0 and swaps the arguments if `start > end`. `substr(start, length)` is a deprecated legacy method; avoid it. → *More:* §6.4 · §6.6 String methods

Q101. How do you reverse a string? What goes wrong with emoji? `s.split('').reverse().join('')` works for plain ASCII but splits emoji and other characters outside the Basic Multilingual Plane, because `split('')` works on UTF-16 **code units** and those characters use two (`'👍'.length` is 2). `[...s].reverse().join('')` splits by **code points** and handles them; for user-perceived characters (flags, combined emoji, accents) use `Intl.Segmenter`. → *More:* §6.5 Unicode

Q102. What are template literals and tagged templates? Template literals are backtick strings with `${expression}` interpolation and real line breaks. A **tagged template** `tag`text ${x}`` calls `tag(strings, ...values)` with the literal parts and the evaluated values separately, so the function can escape, translate or build something else — used for HTML escaping, SQL parameter binding, styled-components and `String.raw` (no escape processing). → *More:* §6.3 Template literals

Q103. What are the common pitfalls of `Date`? What is Temporal? `Date` months are 0-based (`new Date(2026, 0, 1)` is January), objects are mutable, parsing non-ISO strings is unreliable, and date-only ISO strings (`'2026-01-01'`) are read as UTC while date-time strings without an offset are local time. Time zones other than local/UTC are hard. **Temporal** is the replacement API with immutable types (`PlainDate`,

`ZonedDateTime`, `Instant`, `Duration`); it is finished (Stage 4) for ES2027 and ships in Firefox 139+, Chrome/Edge 144+ and Node 26, but not in Node 24 or Safari yet, so use a polyfill there. For formatting, use `Intl.DateTimeFormat`. → *More:* §5.11 `Date`

Q104. How do you generate a random integer between `min` and `max` (inclusive)? `Math.floor(Math.random() * (max - min + 1)) + min`. `Math.random()` returns a float in [0, 1). It is not cryptographically secure: for tokens or IDs use `crypto.getRandomValues()` or `crypto.randomUUID()`. → *More:* §5.10 `Math`

---

6. The event loop, promises and `async` / `await`

Q105. ⋆ JavaScript is single-threaded — so how does it do asynchronous work? Your JavaScript runs on one thread with one **call stack**: one thing at a time. But the **host** (browser or Node) does the slow work — timers, network, file I/O — outside that thread, and when it finishes it queues a callback. The **event loop** runs queued callbacks one by one whenever the call stack is empty. So JavaScript never blocks waiting for I/O, but a long synchronous computation does block everything (clicks, rendering, other requests). → *More:* §12.1 · §12.2 The event loop

Q106. ⋆ Explain the event loop. What are the call stack, the task queue and the microtask queue? The **call stack** holds the functions running right now. The **task (macrotask) queue** holds callbacks from timers, I/O, UI events and messages. The **microtask queue** holds promise reactions (`then` / `catch` / `finally` callbacks, code after `await`), `queueMicrotask` callbacks and `MutationObserver` callbacks. The loop: run one task to completion; then run **all** microtasks, including ones added meanwhile; then (in browsers) render if needed; repeat. So all pending promise callbacks run before the next timer. → *More:* §12.2 · §12.3 Ordering puzzles · §17.4.5 Q35

Q107. ⋆ What is the difference between microtasks and macrotasks? Give examples of each. Microtasks: promise callbacks, `await` continuations, `queueMicrotask`, `MutationObserver`; in Node, `process.nextTick` runs in an even earlier queue. Macrotasks (tasks): `setTimeout`, `setInterval`, `setImmediate` (Node), I/O callbacks, UI events, `MessageChannel` messages. After every task the microtask queue is drained completely — which is why an infinite chain of microtasks can freeze the page just like a `while (true)` loop, while timers yield. → *More:* §12.2

Q108. ⋆ What does `setTimeout(fn, 0)` really do? It queues `fn` as a task to run **at least** 0 ms later — after the current code and all microtasks finish, and after any earlier tasks. Browsers clamp nested timers (more than five levels deep) to at least 4 ms, and background tabs to 1 s or more. The delay is a minimum, never a guarantee. To run something after the current code but before rendering or timers, use `queueMicrotask`. → *More:* §12.5 Timers

Q109. ⋆ What is a promise? What are its states? A promise is an object that stands for the eventual result of an async operation. It is **pending**, then becomes **settled** exactly once: **fulfilled** with a value or **rejected** with a reason (an error); after that it never changes. You react with `then(onFulfilled, onRejected)`, `catch`, `finally`, or `await`. Callbacks attached after settlement still run (asynchronously), which fixes the "missed the event" problem of callbacks. → *More:* §12.7 Promises

Q110. What are the pros and cons of promises over callbacks? Pros: flat chaining instead of nesting, one `catch` for a whole chain, guaranteed async and at-most-once callbacks (no "called twice" or "called synchronously" bugs), composition with `Promise.all` and friends, and `async` / `await` syntax. Cons: one-shot (a stream of events needs callbacks, async iterators or observables), not cancellable by themselves (use `AbortController`), and an unhandled rejection is easy to miss. → *More:* §12.6 · §12.7

Q111. ⋆ How does promise chaining work? `then` always returns a **new** promise. Whatever the callback returns becomes that promise's value; if it returns a promise, the chain waits for it; if it throws, the new promise rejects and control skips to the next `catch`. A `catch` that returns normally **recovers** — the chain continues fulfilled. Forgetting `return` inside a `then` is the classic bug: the chain moves on without waiting. → *More:* §12.7 · Q222 below

Q112. ⋆ What is the difference between `Promise.all`, `allSettled`, `race` and `any`? `all`: fulfils with an array of all values (in input order) when all fulfil; rejects as soon as **one** rejects. `allSettled`: waits for all and gives `{ status, value | reason }` for each — never rejects. `race`: settles like the **first** promise to settle, success or failure. `any`: fulfils with the first **success**; rejects with an

`AggregateError` only if all reject. Typical uses: parallel requests (`all`), batch with partial failures (`allSettled`), timeout (`race`), fastest mirror (`any`). → *More:* §12.8 · §17.4.5 Q42

Q113. Does `Promise.all` cancel the other promises when one rejects? No. A promise is only a result holder; the operations behind the other promises keep running and their results are ignored. To actually stop them, pass one `AbortSignal` to all of them and call `controller.abort()` in the `catch`. → *More:* §12.11 Cancellation

Q114. ⋆ How do `async` and `await` work? An `async` function always returns a promise: its `return` value fulfils it and a `throw` rejects it. `await p` pauses **that function only** until `p` settles, then resumes with the value (or throws the rejection, so `try`/`catch` works). The rest of the function is scheduled as a microtask; the caller and the event loop keep running meanwhile. It is syntax over promises, not threads. → *More:* §12.9 `async` / `await` · §17.4.5 Q36

Q115. ⋆ How do you run async operations in parallel vs in sequence with `await`? `const a = await f(); const b = await g();` is **sequential**: `g` starts only after `f` finishes (total ≈ sum). For independent operations, start both, then await together: `const [a, b] = await Promise.all([f(), g()]);` (total ≈ the slower). For a list processed one by one, use `for…of` with `await`; for a list in parallel with a limit, use a concurrency pool. → *More:* §17.4.5 Q41 · Q264 below

Q116. ⋆ Why doesn't `await` work inside `forEach`? `forEach` calls the async callback and ignores the promise it returns, so the loop finishes immediately and the code after it runs before any callback completes; errors become unhandled rejections. Use `for…of` (sequential) or `await Promise.all(items.map(async (x) => …))` (parallel). → *More:* §17.4.5 Q40

Q117. ⋆ How do you handle errors with promises and `async`/`await`? What is an unhandled rejection? With promises, end each chain with `.catch()`; with `async`/`await`, wrap awaits in `try`/`catch` (and use `finally` for cleanup). A rejection no handler ever catches is an **unhandled rejection**: browsers fire an `unhandledrejection` event and log it; Node prints the error and **exits with code 1** (the default since Node 15). Common causes: a missing `return` inside `then`, an un-awaited async call, `forEach` with async callbacks. → *More:* §12.10 Error handling across async code · §11.5

Q118. How do you cancel a `fetch` request or add a timeout? Create an `AbortController`, pass `controller.signal` to `fetch(url, { signal })`, and call `controller.abort()`; the `fetch` promise rejects with an `AbortError` `DOMException`. For a timeout use `AbortSignal.timeout(5000)` (rejects with a `TimeoutError`), and combine signals with `AbortSignal.any([a, b])`. The same signal works for event listeners, many Node APIs and your own functions. → *More:* §12.11

Q119. What is `queueMicrotask`? How does `process.nextTick` compare with `setImmediate`? `queueMicrotask(fn)` schedules `fn` in the microtask queue, like a resolved promise's `then` but without creating a promise. In Node, `process.nextTick(fn)` runs even sooner — its queue is drained before the promise microtasks. `setImmediate(fn)` is a macrotask that runs in the "check" phase, after I/O polling. Inside an I/O callback, `setImmediate` always runs before `setTimeout(fn, 0)`; in the main module their order is not guaranteed. → *More:* §12.4 Node's event loop · §17.4.5 Q37

Q120. What are `Promise.withResolvers` and `Promise.try`? `Promise.withResolvers()` (ES2024) returns `{ promise, resolve, reject }`, so you can settle a promise from outside an executor — handy for wrapping event-based APIs or building queues. `Promise.try(fn)` (ES2025) calls `fn` immediately and always returns a promise: a sync return fulfils it and a sync throw rejects it, so sync and async errors go to the same `catch`. → *More:* §12.7

Q121. When does a promise executor run? What if `resolve` is called twice? The executor function passed to `new Promise` runs **synchronously**, immediately, during the constructor. Only the first call to `resolve` or `reject` counts; later calls are ignored silently. Throwing inside the executor rejects the promise. → *More:* §17.4.5 Q38

Q122. What is `for await…of`? A loop over an **async iterable** — something whose `next()` returns promises, like an async generator, a Node readable stream, or paginated API results. Each iteration awaits the next value. It also accepts an array of promises, awaiting them in order (but they already run in parallel). `Array.fromAsync` (ES2026, in Node 24) collects one into an array. → *More:* §12.12

Q123. What is the difference between promises and observables? A promise delivers **one** value, is **eager** (the work started before you got it) and cannot be cancelled by itself. An observable (RxJS) delivers **zero to many** values over time, is **lazy** (nothing runs until someone subscribes), can be cancelled by

unsubscribing, and has operators like `map`, `debounceTime`, `switchMap`. Observables suit event streams; promises suit single requests.

Q124. What is the difference between synchronous and asynchronous functions? A synchronous function does all its work before returning; the caller waits, and the thread is blocked meanwhile. An asynchronous function starts the work and returns at once (usually a promise); the result arrives later through a callback or promise, while the thread stays free for other events. → *More:* §12.1

---

7. Errors, modules and tooling

Q125. ⋆ How do `try` / `catch` / `finally` work? What if `finally` has a `return`? `try` runs code; if it throws, `catch (err)` runs (the binding is optional since ES2019); `finally` always runs afterwards — after a normal finish, a `return`, a `break` or an exception — for cleanup. A `return` (or `throw`) in `finally` **overrides** the one from `try` / `catch`, and swallows a pending exception, so never return from `finally`. `try` / `catch` only catches errors thrown synchronously inside it or from awaited promises. → *More:* §11.2 · Q238 below

Q126. What are the built-in error types? `Error` (base), `SyntaxError` (invalid code or `JSON.parse` input), `ReferenceError` (unknown name, TDZ), `TypeError` (wrong kind of value: calling a non-function, reading a property of `undefined`), `RangeError` (value out of range, stack overflow), `URIError`, `EvalError`, and `AggregateError` (several errors, from `Promise.any`). Every error has `name`, `message` and a non-standard but universal `stack`. → *More:* §11.1

Q127. How do you create a custom error class? Extend `Error`, call `super(message, { cause })`, set `this.name`, and add any extra fields. Callers can then use `instanceof` or check `name` / `code`.

```
class HttpError extends Error {
  constructor(status, message, options) { super(message, options); this.name = 'HttpError'; this.status = status;
}
}
throw new HttpError(404, 'User not found', { cause: originalError });
```

→ *More:* §11.3 Custom errors · §11.4 `cause`

Q128. Why doesn't `try` / `catch` catch an error thrown inside a `setTimeout` callback? By the time the callback runs, the `try` block has finished and its stack frame is gone; the callback runs later from the event loop with a fresh stack. Put the `try` / `catch` inside the callback, or wrap the timer in a promise and `await` it. → *More:* §17.4.5 Q39

Q129. How do you catch errors globally? Browser: `window.addEventListener('error', …)` for uncaught exceptions (and failed resource loads, with capture) and `window.addEventListener('unhandledrejection', …)` for promises. Node: `process.on('uncaughtException')` and `process.on('unhandledRejection')` — use them to log and then exit, because the process may be in an unknown state; restart via a process manager. Error monitoring services (like Sentry) hook into these. → *More:* §11.5 · §12.10

Q130. What tools and techniques do you use to debug JavaScript? `console.log` / `table` / `dir` / `trace` / `time`; the `debugger` statement; DevTools breakpoints (line, conditional, logpoints, DOM-change, XHR/fetch, exception breakpoints), the call stack and scope panes, the Network and Performance panels; source maps (files that map built code back to the original source) to debug bundled code; `node --inspect` with Chrome DevTools or VS Code for Node; and a failing test that reproduces the bug. → *More:* §11.7 · §11.9

Q131. What are the performance implications of `try` / `catch`? In modern engines a `try` / `catch` costs essentially nothing when no error is thrown. **Throwing** is expensive (an error object and stack trace are built), so don't use exceptions for normal control flow in hot loops — return a value instead.

Q132. What is `using` (explicit resource management)? `using x = resource` declares a variable whose `[Symbol.dispose]()` method is called automatically when the block ends, even after an exception; `await using` calls `[Symbol.asyncDispose]()`. It replaces `try` / `finally` cleanup for files, locks and connections. The proposal is finished for ES2027 and already works in Node 24. → *More:* §11.6 Guaranteed cleanup

Q133. ⋆ What is the difference between ES modules and CommonJS? CommonJS (Node's original system): `require()` / `module.exports`, loaded synchronously at run time, exports are a copied value, `require` can be called anywhere. **ES modules** (the standard): `import` / `export` declared statically at the top, so tools

know the dependency graph before running (tree shaking, browser support); loaded asynchronously; exports are **live bindings**; strict mode always; top-level `await`; `this` is `undefined` at top level. Node picks the system by extension (`.mjs` / `.cjs`) or the `"type"` field in `package.json`. ESM can import CJS; Node 22.12+ can also `require()` ESM unless the module uses top-level `await`. → *More:* §10.8 CommonJS vs ES modules in Node

Q134. Named exports vs default exports? A module can have many named exports (`export const a`, imported as `import { a } from`) and at most one default (`export default`, imported under any name). Named exports keep names consistent across the codebase, are easier for auto-import and refactoring tools, and catch typos at link time; many teams prefer them. Default exports are common for "one main thing per file" (React components, config). → *More:* §10.2

Q135. What are live bindings in ES modules? An import is a read-only view of the exporting module's variable, not a copy. If the exporter later changes `export let count`, every importer sees the new value. Importers cannot assign to it (`TypeError`). In CommonJS, `const { count } = require(…)` copies the value at that moment. → *More:* §10.3

Q136. What is dynamic `import()`, and what are its use cases? `import('./module.js')` loads a module at run time and returns a promise of its namespace object. It works in CommonJS and classic scripts too. Uses: **code splitting** (load a heavy chart library only when the chart tab opens), conditional or locale-based loading, lazy routes in SPAs (`React.lazy`), loading ESM from CommonJS. Non-JavaScript modules need an **import attribute** (ES2025): `import config from './config.json' with { type: 'json' };` or `import('./config.json', { with: { type: 'json' } })`. → *More:* §10.4

Q137. What is top-level `await`? In an ES module you can use `await` outside any function (ES2022). The module's evaluation pauses until it settles, and modules that import it wait too — so use it for setup (loading config, a DB connection, a dynamic import), not for slow work. Before it, the workaround was an async IIFE. A module using it cannot be loaded with `require()`. → *More:* §10.5 Top-level `await`

Q138. What happens with circular imports? ES modules allow cycles: the graph is linked first, then evaluated depth-first, so one module in the cycle runs before the other has finished; reading a not-yet-initialised `let` / `const` / `class` export throws a TDZ `ReferenceError` (function declarations are hoisted and work). In CommonJS, the second module gets the first one's **partially filled** `module.exports`. Fix by moving shared code into a third module or deferring access into functions. → *More:* §10.3

Q139. How do you get `__dirname` and `__filename` in an ES module? They don't exist in ESM. Use `import.meta.dirname` and `import.meta.filename` (Node 20.11+), or `fileURLToPath(import.meta.url)`. `import.meta.url` is the module's own `file://` URL; `new URL('./data.json', import.meta.url)` resolves a file next to the module. → *More:* §10.6 `import.meta` and JSON modules · §15.5

Q140. What is tree shaking? A bundler optimisation that removes exports nobody imports. It relies on ES modules' **static** `import` / `export` (the bundler sees exactly what is used) and on modules being free of side effects at import time (`"sideEffects": false` in `package.json` tells the bundler it may drop unused files). CommonJS, `import * as` used dynamically, and code with top-level side effects defeat it. → *More:* §10.11 Bundlers

Q141. What do bundlers, transpilers and polyfills do? Shim vs polyfill? A **bundler** (Vite, webpack, esbuild, Rollup) follows imports and produces a few optimised files: minified, split, tree-shaken, with source maps. A **transpiler** (Babel, TypeScript, SWC, esbuild) rewrites new **syntax** into older syntax (or TS into JS). A **polyfill** adds a missing **API** at run time using existing features (`Array.prototype.at` written in JS). A **shim** is the broader term: any library that fills in or adapts an API, possibly with a different interface. → *More:* §10.11 · §10.12 Transpilers and polyfills

Q142. What do `^` and `~` mean in `package.json`? Why commit the lockfile? Versions follow **semver**: `MAJOR.MINOR.PATCH` (breaking, feature, fix). `^1.4.2` allows `>=1.4.2 <2.0.0` (any non-breaking update); `~1.4.2` allows `>=1.4.2 <1.5.0` (patches only). (For `0.x`, `^0.4.2` means `<0.5.0`.) The lockfile (`package-lock.json`) records the exact resolved version of every package in the tree, so every install and CI build gets identical code; use `npm ci` in CI. → *More:* §10.10 Semantic versioning, ranges and lockfiles

Q143. What are minification and obfuscation? Minification removes whitespace and comments and shortens local names to make files smaller and faster to download; it is a normal build step (Terser, esbuild). **Obfuscation** deliberately makes code hard to read to deter copying; it is not security — anything shipped to the browser can be read — and it is not encryption, which needs a key to reverse. → *More:* §10.11

8. The DOM and events

Q144. What is the DOM? What is the difference between `window` and `document`? The DOM (Document Object Model) is the browser's live tree of objects representing the HTML page; JavaScript reads and changes the page through it. `window` is the global object for the browser tab: timers, `location`, `history`, `localStorage`, viewport size — these browser-level APIs are called the BOM (Browser Object Model). `document` (`window.document`) is the root of the DOM tree for the loaded page. → *More:* §13.1 The DOM tree

Q145. ⋆ Explain event bubbling and capturing (the event flow). A DOM event travels in three phases: **capture** — from `window` down through the ancestors to the target; **target** — at the element itself; **bubble** — back up from the target to `window`. Listeners run in the bubble phase by default; pass `{ capture: true }` to run in the capture phase. Most events bubble; some don't (`focus`, `blur`, `mouseenter`, `load` — use `focusin` / `focusout` to get bubbling focus events). → *More:* §13.6 Propagation

Q146. ⋆ What is event delegation, and why use it? Instead of a listener on every child, put one listener on a common ancestor and use `event.target.closest(selector)` to find which child was hit. It works because events bubble. Benefits: one listener instead of thousands (less memory), and it automatically works for children added later. Check the matched element is inside the container, and remember non-bubbling events need capture.

```
list.addEventListener('click', (e) => {
  const btn = e.target.closest('button.delete');
  if (btn && list.contains(btn)) removeItem(btn.dataset.id);
});
```

→ *More:* §13.7 Event delegation

Q147. ⋆ What is the difference between `event.target` and `event.currentTarget`? `target` is the element where the event originated (the innermost element clicked). `currentTarget` is the element whose listener is running now — the one you called `addEventListener` on. With delegation they differ; `currentTarget` is `null` after the handler returns (so log it inside the handler). → *More:* §13.6

Q148. ⋆ `preventDefault()` vs `stopPropagation()` vs `stopImmediatePropagation()` vs `return false`? `preventDefault()` cancels the browser's **default action** (following a link, submitting a form, checking a checkbox) but the event still propagates. `stopPropagation()` stops the event moving to further elements, but other listeners on the current element still run. `stopImmediatePropagation()` also stops those. `return false` does nothing in `addEventListener`; only in an inline `onclick` property handler does it act like `preventDefault()` (and in jQuery, like both). → *More:* §13.8 Default actions and `preventDefault` · §13.6

Q149. What is the difference between an attribute and a property? An **attribute** is written in the HTML and lives on the element as a string (`getAttribute('value')`). A **property** is a field of the DOM object (`input.value`), with a real type. Many are linked at first, but some diverge: for an `<input>`, the `value` attribute is the **initial** value while the `value` property is the **current** text the user typed. `data-*` attributes map to `element.dataset`. → *More:* §13.4

Q150. ⋆ What is the difference between `DOMContentLoaded` and `load`? Between `async` and `defer` scripts? `DOMContentLoaded` fires when the HTML is parsed and the DOM is built (deferred scripts have run); `load` fires later, when images, stylesheets and iframes have also finished. A plain `<script>` blocks parsing while it downloads and runs. `defer` downloads in parallel and runs after parsing, in document order — the right default for app scripts. `async` downloads in parallel and runs as soon as it arrives, in any order — for independent scripts like analytics. Module scripts are deferred by default. → *More:* §13.12 When scripts run

Q151. `querySelectorAll` vs `getElementsByClassName`: what is the difference? `querySelectorAll` takes any CSS selector and returns a **static** `NodeList` (a snapshot; later DOM changes don't affect it), which has `forEach`. `getElementsByClassName` / `getElementsByTagName` return a **live** `HTMLCollection` that updates as the DOM changes (so looping while removing elements can skip items) and has no `forEach`. `querySelector` returns the first match or `null`. → *More:* §13.2 Selecting and traversing

Q152. ⋆ `innerHTML` vs `textContent` vs `innerText`? `innerHTML` reads/writes the element's HTML markup — setting it parses HTML, so untrusted input causes XSS, and it destroys and recreates child nodes (losing listeners). `textContent` reads/writes raw text of all descendants, including hidden ones, and

never parses HTML — use it for user data. `innerText` returns the **rendered** text (respects CSS, skips hidden elements) and forces a layout to compute it, so it is slower. → *More:* §13.3 · §14.10 XSS

Q153. How do you add many elements to the page efficiently? Build them off-page and insert once: append to a `DocumentFragment` (a lightweight container whose children move into the DOM in one operation) or call `parent.append(...nodes)` / `replaceChildren(...nodes)` once. This causes one layout instead of one per element. For very long lists, render only the visible rows (virtualisation). → *More:* §13.3 · §13.11

Q154. ⋆ What is the difference between reflow and repaint? What is layout thrashing? Reflow (layout) recalculates the size and position of elements — triggered by changing geometry (width, font, adding nodes). **Repaint** redraws pixels without layout changes (colour, visibility). Reflow is more expensive and may cascade. **Layout thrashing** is interleaving DOM writes with reads of layout properties (`offsetHeight`, `getBoundingClientRect`) in a loop, which forces a synchronous reflow each time. Fix: batch all reads, then all writes; animate `transform` / `opacity` (often handled by the compositor without layout or paint); use `requestAnimationFrame`. → *More:* §13.11

Q155. Why remove event listeners? How? A listener keeps its callback — and everything the callback closes over — alive while the target exists, so listeners on long-lived targets (`window`, `document`) from removed components leak memory and keep running. Remove with `removeEventListener(type, sameFunctionReference, sameCaptureFlag)` (an inline arrow can't be removed), use `{ once: true }` for one-shot listeners, or pass `{ signal }` and call `controller.abort()` to remove many at once. → *More:* §13.5 Events and listeners

Q156. What are passive event listeners? `addEventListener('touchstart', fn, { passive: true })` promises the browser that `fn` will not call `preventDefault()`, so it can start scrolling immediately instead of waiting for the handler. Browsers make `touchstart` / `touchmove` / `wheel` listeners on `window`, `document` and `body` passive by default. → *More:* §13.5

Q157. How do you create and dispatch a custom event? `el.dispatchEvent(new CustomEvent('cart:add', { detail: { id: 7 }, bubbles: true }))`, and listen with `addEventListener('cart:add', (e) => e.detail.id)`. Set `bubbles: true` if ancestors should hear it (custom events don't bubble by default). Dispatch is **synchronous**: listeners run before `dispatchEvent` returns. Plain `EventTarget` can be subclassed for non-DOM event buses. → *More:* §13.5

Q158. What is `MutationObserver`? An API that calls your callback (as a microtask, batched) when a DOM subtree changes: children added/removed, attributes changed, text changed. It replaced the slow, synchronous mutation events. Uses: reacting to DOM changes made by third-party code, auto-initialising widgets inserted later. Call `disconnect()` when done. → *More:* §13.10

Q159. How do you handle forms with JavaScript (submit, validation)? Listen for `submit` on the form (not `click` on the button, so the Enter key works too), call `preventDefault()` to stop the page reload, read values with `new FormData(form)`, and send them with `fetch`. Use HTML constraints (`required`, `type="email"`, `pattern`, `min`) and the Constraint Validation API (`checkValidity()`, `reportValidity()`, `setCustomValidity()`, `input.validity`) before custom code. Always validate again on the server. → *More:* §13.9 Forms

Q160. What are Web Components and the Shadow DOM? Web Components are the browser's native component model: **custom elements** (`class MyCard extends HTMLElement`, registered with `customElements.define('my-card', MyCard)`), **Shadow DOM** (`this.attachShadow({ mode: 'open' })` — a hidden subtree whose styles and IDs are isolated from the page), and `<template>` / `<slot>` for content. They work with any framework or none.

---

9. Browser APIs, storage, networking and security

Q161. ⋆ What are the differences between cookies, `localStorage` and `sessionStorage`? Cookies (~4 KB each) are sent to the server with every matching request; they can be `HttpOnly` (invisible to JavaScript), `Secure`, `SameSite`, and expire when you choose — use them for sessions. **`localStorage`** (~5 MB per origin, strings only, synchronous) persists until cleared and is never sent automatically. **`sessionStorage`** is the same but per tab, cleared when the tab closes. JavaScript can read both storages, so never put tokens or secrets there if XSS is possible. For large or structured data use IndexedDB. → *More:* §14.3 Web Storage · §14.4 Cookies and IndexedDB

Q162. What is the `storage` event? When one tab changes `localStorage`, every **other** tab of the same origin receives a `storage` event on `window` with `key`, `oldValue`, `newValue`. It is a simple way to sync tabs (e.g. log out everywhere); `BroadcastChannel` is the more general tool. → *More:* §14.3

Q163. What is IndexedDB, and when would you use it? An asynchronous, transactional key-value database in the browser that stores structured data (objects, files, blobs) with indexes, and can hold far more than `localStorage` (a share of free disk space). Use it for offline apps, caching API data, and drafts. Its raw API is event-based and verbose, so most people use a small promise wrapper. → *More:* §14.4

Q164. What do the cookie attributes `HttpOnly`, `Secure` and `SameSite` do? `HttpOnly` hides the cookie from `document.cookie`, so an XSS script cannot steal it. `Secure` sends it only over HTTPS. `SameSite` controls sending on cross-site requests: `Strict` never, `Lax` (the browsers' default when the attribute is missing, in Chrome and Edge) only on top-level navigations with safe methods like a link click, `None` always (requires `Secure`). Also: `Max-Age`/`Expires`, `Path`, `Domain`; the `__Host-` prefix forces `Secure`, `Path=/` and no `Domain`. → *More:* §14.4

Q165. What is the same-origin policy? A browser rule: a page may only **read** responses and data from the same **origin** — the same scheme, host and port (`https://app.com:443`). Other origins can still be sent requests (forms, images, scripts, `fetch`) but scripts cannot read the response unless the server allows it with CORS. It stops a malicious site from reading your bank's pages using your cookies. → *More:* §14.2

Q166. ⋆ What is CORS? What is a preflight request? CORS (Cross-Origin Resource Sharing) lets a **server** opt in to cross-origin reads via response headers like `Access-Control-Allow-Origin: https://app.com`. For "non-simple" requests — methods other than GET/HEAD/POST, custom headers such as `Authorization`, or `Content-Type: application/json` — the browser first sends an `OPTIONS` **preflight** asking permission, and only sends the real request if the server answers with matching `Access-Control-Allow-*` headers. With cookies, the server must send `Access-Control-Allow-Credentials: true` and an exact origin (not `*`). CORS is enforced by the browser; it is not server security, and fixing a CORS error means changing the server (or using a same-origin proxy). → *More:* §14.2

Q167. ⋆ What is XSS, and how do you prevent it? Cross-site scripting: an attacker gets their script to run in your page — **stored** (saved in the DB and shown to others), **reflected** (in a URL echoed back) or **DOM-based** (your JS writes untrusted data into the page). The script can then do anything the user can, including reading tokens. Prevention: insert user data as text (`textContent`, framework escaping), never into `innerHTML`/`eval`/`href="javascript:"`; sanitise HTML you must render with DOMPurify; set a strict **Content Security Policy**; keep session cookies `HttpOnly`; consider Trusted Types. → *More:* §14.10

Q168. ⋆ What is CSRF, and how do you prevent it? Cross-site request forgery: a malicious site makes the victim's browser send a request (e.g. a hidden form POST) to your site, and the browser attaches the victim's cookies, so the action runs as them. Prevention: `SameSite=Lax` or `Strict` cookies; a **CSRF token** (a random value in the page that the server checks on every state-changing request); checking the `Origin`/`Sec-Fetch-Site` header; never changing state on GET. APIs authenticated with an `Authorization` header (not cookies) are not exposed to CSRF. → *More:* §14.11

Q169. What is a Content Security Policy (CSP)? An HTTP response header (`Content-Security-Policy`) that tells the browser which sources of scripts, styles, images and connections are allowed. A strict policy (scripts only from your origin or with a per-response **nonce**, no inline scripts, no `eval`) means that even if an attacker injects markup, their script won't run. It is defence in depth against XSS; also `frame-ancestors` blocks clickjacking. → *More:* §14.12

Q170. Where should you store a JWT or auth token in the browser? The safest common choice is an `HttpOnly; Secure; SameSite` cookie set by the server: JavaScript (and therefore XSS) cannot read it, and `SameSite` plus CSRF tokens handle CSRF. `localStorage` is easy but any XSS can steal the token. Another pattern keeps a short-lived access token **in memory** and a refresh token in an `HttpOnly` cookie. Either way, XSS prevention comes first. → *More:* §14.4

Q171. What is clickjacking? What is prototype pollution? Clickjacking: your page is loaded in an invisible iframe on an attacker's site so the user clicks your buttons unknowingly; block framing with `Content-Security-Policy: frame-ancestors 'self'` (or `X-Frame-Options: DENY`). **Prototype pollution**: code that deep-merges untrusted JSON writes a key like `__proto__` or `constructor.prototype`, adding properties to `Object.prototype` and so to every object. Prevent it by rejecting those keys, using `Object.create(null)` or `Map` for dictionaries, and validating input against a schema. → *More:* §14.12 · §17.4.4 Q32

Q172. ⋆ How does `fetch` work? Does it reject on a 404? `fetch(url, options)` returns a promise of a `Response`. It rejects **only on network failure** (or abort/CORS failure); HTTP errors like 404 or 500 still fulfil, so check `response.ok` (status 200–299) or `response.status`. Reading the body is a second async step: `await res.json()`, `text()`, `blob()`; the body can be read only once. Options: `method`, `headers`, `body` (stringify JSON yourself), `credentials`, `signal`. Unlike `XMLHttpRequest`, it is promise-based and supports streaming.

```
const res = await fetch('/api/users', { signal: AbortSignal.timeout(5000) });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

→ *More:* §14.1 `fetch` and HTTP

Q173. What are Web Workers? What can't they do? A Web Worker runs a script on a separate background thread, so heavy computation (parsing, image processing) doesn't freeze the UI. It communicates with the page by `postMessage` (data is copied using structured clone, or **transferred** for `ArrayBuffer`s). Workers have no access to the DOM, `window` or `localStorage`, but they can use `fetch`, timers, IndexedDB and WebSockets. → *More:* §14.8 Web Workers

Q174. What is a service worker? What is a PWA? A service worker is a background script that sits between your page and the network: it can intercept `fetch` requests and answer from a cache (offline support), receive push messages and do background sync. It has no DOM access, requires HTTPS, and has its own lifecycle (install, activate, then controls pages). A **PWA** (progressive web app) is a website with a service worker and a web app manifest, so it can work offline and be installed like an app. → *More:* §14.8

Q175. How does client-side routing work in an SPA? A single-page application changes the URL without a page reload using the **History API**: `history.pushState(state, '', '/path')` adds an entry and `replaceState` changes the current one; the app then renders the view for the new path. Back/forward fire a `popstate` event, which the app handles by rendering the matching view. The server must return the app's HTML for every route so direct links work. → *More:* §14.7 The History API

Q176. How do you lazy-load images or implement infinite scroll? Use `IntersectionObserver`: it calls you asynchronously when a target element enters or leaves the viewport (with a `rootMargin` to start early), without scroll listeners or layout reads. For images, `<img loading="lazy">` is built in. For infinite scroll, observe a sentinel element after the last item and load the next page when it becomes visible (with a guard so only one load runs at a time). → *More:* §14.9

Q177. What is `requestAnimationFrame`? `requestAnimationFrame(cb)` runs `cb` just before the browser's next repaint (usually 60 times a second, matching the display), with a high-resolution timestamp. Use it for animations and DOM writes tied to frames instead of `setTimeout`: it is synchronised with rendering and paused in background tabs. → *More:* §14.5

Q178. How do you read query-string parameters? `new URL(location.href).searchParams.get('q')` or `new URLSearchParams(location.search).get('q')`. `URLSearchParams` also builds query strings with correct encoding (`params.set('q', 'a b&c')`). Manually, use `encodeURIComponent` for a single value; `encodeURI` is for a whole URL and leaves `&`, `?`, `/` alone. → *More:* §14.6 `URL` and `URLSearchParams`

Q179. What is `postMessage`, and how do you use it safely? `otherWindow.postMessage(data, targetOrigin)` sends data between windows, iframes or workers of different origins. Always pass an exact `targetOrigin` (not `'*'`) so data isn't delivered to the wrong site, and in the `message` handler check `event.origin` against an allow-list and validate `event.data` before using it.

Q180. Polling vs long polling vs WebSockets vs Server-Sent Events? Polling asks the server every few seconds (simple, wasteful, delayed). **Long polling** keeps a request open until there is news, then reconnects. **Server-Sent Events** (`EventSource`) is a one-way server-to-browser stream over plain HTTP with automatic reconnect — good for notifications and feeds. **WebSockets** give a persistent two-way connection — good for chat, games, collaboration. → *More:* §14.1

Q181. What is feature detection, and why prefer it over user-agent sniffing? Feature detection checks whether the API exists before using it (`if ('IntersectionObserver' in window)`, `CSS.supports(…)`), then uses it or a fallback/polyfill. User-agent sniffing guesses features from the browser name string, which is easy to spoof, frozen or reduced in modern browsers, and wrong as soon as a browser adds the feature. → *More:* §10.12

10. Performance and memory

Q182. How does garbage collection work in JavaScript? Memory is freed automatically. Engines use **reachability**: starting from roots (globals, the current call stack, active closures), everything reachable through references is kept; everything else is garbage (**mark-and-sweep**). Cycles are not a problem — two objects pointing at each other but unreachable are collected. V8 is **generational**: new objects go into a small young area collected often and cheaply; survivors are promoted to the old area, collected less often. → *More:* §17.4.7 Q51

Q183. ⋆ What causes memory leaks in JavaScript? How do you find them? Something keeps references alive that you no longer need: forgotten event listeners and `setInterval`s, growing caches/maps with no limit, closures holding large data, detached DOM nodes still referenced from JS, accidental globals, unresolved promises holding resources. Find them with DevTools Memory: take heap snapshots before and after repeating an action and compare, look for detached elements, or use the allocation timeline. Fix by cleaning up (remove listeners, clear timers, `AbortController`), bounding caches (LRU), and using `WeakMap`. → *More:* §17.4.7 Q52

Q184. What are `WeakRef` and `FinalizationRegistry`? `WeakRef(obj)` holds a reference that doesn't stop garbage collection; `deref()` returns the object or `undefined` once it's gone — useful for caches of large objects. `FinalizationRegistry` lets you register a callback that may run after an object is collected — for cleaning external resources. Both are non-deterministic (GC timing is unpredictable, callbacks may never run), so don't base program logic on them. → *More:* §8.3 · §17.4.7 Q53

Q185. ⋆ What is the difference between debouncing and throttling? When do you use each? Debounce waits until calls stop for a quiet period, then runs once — for search-as-you-type, auto-save, resize-end. **Throttle** runs at most once per interval while calls continue — for scroll position, mouse move, rate-limited API calls, infinite-scroll checks. Both reduce how often an expensive handler runs. → *More:* §17.3 Debounce · §17.3 Throttle

Q186. ⋆ How would you improve the performance of a slow web page's JavaScript? Measure first (Lighthouse, Performance panel, Core Web Vitals). Then: ship less JS (code splitting, lazy loading with dynamic `import()`, tree shaking, drop heavy libraries); load scripts with `defer`; break **long tasks** (anything over 50 ms blocks input) into chunks with `scheduler.yield()` or `setTimeout`, or move them to a Web Worker; avoid layout thrashing; debounce/throttle frequent handlers; use event delegation; virtualise long lists; cache network results; memoise expensive pure computations. → *More:* §13.11 · §10.11

Q187. What are Core Web Vitals? Google's user-centred metrics: **LCP** (Largest Contentful Paint — loading; good ≤ 2.5 s), **INP** (Interaction to Next Paint — responsiveness to clicks and keys; good ≤ 200 ms; it replaced FID in 2024), and **CLS** (Cumulative Layout Shift — visual stability; good ≤ 0.1). JavaScript mostly hurts INP (long tasks on the main thread) and LCP (render-blocking or client-rendered content).

Q188. How do you measure the performance of code? `performance.now()` gives a high-resolution timestamp for timing a piece of code; `performance.mark`/`measure` create named entries visible in DevTools; `console.time`/`timeEnd` for quick checks. For pages, use the Performance panel and Lighthouse; for real users, `PerformanceObserver` (long tasks, paint and web-vitals entries). In Node, `node --cpu-prof` or `--inspect` profiles. Benchmark with realistic data and warm-up, since the JIT optimises hot code. → *More:* §11.7 · §11.9

Q189. How does V8 run your code fast? What are hidden classes and inline caching? V8 first interprets bytecode (Ignition), collects type feedback, then compiles hot functions to optimised machine code (TurboFan, with intermediate tiers) — **JIT** (just-in-time) compilation. Objects get **hidden classes** (shapes) describing their property layout; objects created with the same properties in the same order share a shape. **Inline caches** remember the shape seen at each property access, making repeat accesses fast. So: initialise all properties in the constructor in the same order, don't `delete` properties, keep functions monomorphic (called with one shape of argument). If assumptions break, V8 **deoptimises** back to slower code. → *More:* §1.3 JavaScript engines

Q190. How do you render a list of 100,000 items without freezing the page? Don't create 100,000 DOM nodes: **virtualise** — render only the rows in (and just around) the viewport and reposition them on scroll (libraries: TanStack Virtual, react-window), or paginate. If data processing is heavy, do it in

a Web Worker or in chunks that yield to the event loop. CSS `content-visibility: auto` lets the browser skip rendering off-screen sections.

Q191. A page freezes while the user types in a search box. How do you debug and fix it? Record the interaction in the Performance panel and look for long tasks on the main thread. Usual causes: filtering a huge list or calling an API on every keystroke, and re-rendering everything. Fixes: debounce the input; cancel stale requests with `AbortController` (and ignore out-of-order responses); memoise the filtering; virtualise the result list; move heavy matching to a worker. → *More:* §12.13 Real-world async patterns

---

11. Node.js

Q192. ⋆ What is Node.js? Is it single-threaded? Node is a JavaScript runtime outside the browser: Google's V8 engine plus **libuv** (a C library providing the event loop, async I/O and a thread pool) plus core modules (`fs`, `http`, `crypto`, `stream`…). Your JavaScript runs on one main thread, but libuv uses the OS's async facilities for network I/O and a **thread pool** (4 threads by default, `UV_THREADPOOL_SIZE`) for file system, DNS lookup, `crypto` and `zlib` work. With `worker_threads` you can run JavaScript on extra threads too. → *More:* §15.1 · §15.2

Q193. ⋆ Explain the phases of Node's event loop. Each turn goes through phases: **timers** (`setTimeout`/`setInterval` callbacks that are due), **pending callbacks** (some deferred system errors), **idle/prepare** (internal), **poll** (wait for and run I/O callbacks), **check** (`setImmediate` callbacks), **close callbacks** (`'close'` events). Between every callback, Node drains the `process.nextTick` queue, then the promise microtask queue. → *More:* §12.4 Node's event loop

Q194. ⋆ Why is blocking the event loop bad in Node? How do you handle CPU-heavy work? One thread serves every request; a 2-second synchronous loop (big JSON parse, image resize, `fs.readFileSync` in a request handler, a catastrophic regex) delays **all** other clients by 2 seconds. Fixes: use async APIs; split work into chunks; move CPU work to `worker_threads` or a separate service/queue; scale across cores with multiple processes (cluster, containers). → *More:* §15.2

Q195. `worker_threads` vs `child_process` vs `cluster`? `worker_threads`: extra JavaScript threads in the same process, each with its own V8 and event loop, communicating by messages or `SharedArrayBuffer` — for CPU-heavy JS. `child_process` (`spawn`, `exec`, `fork`): separate OS processes, for running other programs or isolating work. `cluster`: forks several Node processes that share one server port to use all CPU cores (often replaced by a process manager or container orchestration). → *More:* §15.2 · §15.11

Q196. ⋆ What are streams? What is backpressure? A stream processes data piece by piece (chunks) instead of loading it all into memory. Types: **Readable** (file read, HTTP request), **Writable** (file write, HTTP response), **Duplex** (TCP socket) and **Transform** (gzip). **Backpressure** happens when the reader is faster than the writer: `write()` returns `false` and you should wait for `'drain'`. `pipeline(src, transform, dest)` from `node:stream/promises` handles backpressure, errors and cleanup for you — use it instead of `.pipe()`.

```
import fs from 'node:fs';
import zlib from 'node:zlib';
import { pipeline } from 'node:stream/promises';
await pipeline(fs.createReadStream('big.log'), zlib.createGzip(), fs.createWriteStream('big.log.gz'));
```

→ *More:* §15.9 Streams

Q197. What is a `Buffer`? A `Buffer` is Node's fixed-size chunk of raw bytes (a subclass of `Uint8Array`), used for binary data: files, network packets, images, crypto. Convert with `Buffer.from('hi', 'utf8')` and `buf.toString('base64')`. `Buffer.alloc(n)` gives zero-filled memory; `allocUnsafe` is faster but may contain old data. → *More:* §15.7

Q198. What is `EventEmitter`? Node's built-in publish/subscribe class (`node:events`): `on(event, fn)` registers listeners, `emit(event, ...args)` calls them **synchronously** in registration order, `once` listens once, `off` removes. Streams, servers and sockets are emitters. Emitting `'error'` with no listener throws, so always handle `'error'`. Warns after 10 listeners per event (a common leak hint). `events.once(emitter, 'x')` gives a promise. → *More:* §15.8 Events: `EventEmitter` · Q258 below

Q199. How do you read command-line arguments and environment variables? What do exit codes mean? `process.argv` is the array `[nodePath, scriptPath, ...args]` (`util.parseArgs` parses flags). `process.env.NAME` reads environment variables (always strings); Node can load a `.env` file with `--env-file=.env`. Exit code `0` means success, anything else failure; set `process.exitCode = 1` and let the process end naturally, rather than `process.exit()`, which cuts off pending I/O. → *More:* §15.6 `process`

Q200. `fs.readFileSync` vs `fs.readFile` vs `fs/promises`? `readFileSync` blocks the thread until the file is read — fine in startup scripts and CLIs, bad inside a server request. `fs.readFile(path, cb)` is the callback async version; `import { readFile } from 'node:fs/promises'` is the promise version for `await` — the modern default. For large files use streams. → *More:* §15.4 Files with `node:fs/promises`

Q201. How do you create an HTTP server in Node without a framework? `http.createServer((req, res) => {...}).listen(3000)`. The handler reads `req.method`, `req.url` (parse with `new URL(req.url, 'http://localhost')`), headers and the body stream, then sets `res.statusCode`, headers and calls `res.end(body)`. Frameworks (Express, Fastify, Hono) add routing, middleware (functions that run in a chain before your handler), body parsing and error handling on top. → *More:* §15.10

Q202. What is middleware (as in Express)? A function `(req, res, next)` in a pipeline that runs for each request: it can read or modify the request/response, end the response, or call `next()` to pass control on (`next(err)` jumps to error-handling middleware, which has four parameters). Used for logging, auth, body parsing, CORS, rate limiting. Order of registration is the order of execution. → *More:* §15.10

Q203. What is the difference between `dependencies`, `devDependencies` and `peerDependencies`? `npm` vs `npx`? `dependencies` are needed at run time; `devDependencies` only for development and builds (test runners, bundlers, types); `peerDependencies` are packages a library expects the host app to provide (a React component library and `react`). `npm` installs and manages packages and runs scripts; `npx` runs a package's binary without a global install. → *More:* §10.9 npm and `package.json`

Q204. How does JavaScript in Node differ from JavaScript in the browser? Same language and engine family; different host APIs. Node has no DOM, `window` or `document`, but has file system, processes, sockets, `Buffer`, `require` and full OS access; the browser is sandboxed and has the DOM, `localStorage`, rendering. The global object is `global` vs `window` (`globalThis` in both). Many web APIs now exist in both: `fetch`, `URL`, `AbortController`, `structuredClone`, web streams, `crypto.subtle`. → *More:* §15.1

Q205. How do you scale a Node.js application? Keep each process non-blocking (async I/O, offload CPU work). Run several stateless processes per machine (one per core) behind a load balancer, and several machines horizontally; keep state (sessions, caches) in shared stores like Redis or a database. Use queues for slow background jobs, streams for big payloads, caching and connection pooling for the database. → *More:* §15.2 · System Design Fundamentals

Q206. What are Deno and Bun? Alternative JavaScript runtimes. **Deno** (by Node's creator) uses V8 and Rust, is secure by default (explicit permissions for files and network), runs TypeScript natively and bundles a formatter, linter and test runner. **Bun** uses JavaScriptCore (Safari's engine) and Zig, aims for speed, and bundles a package manager, bundler and test runner. Both are largely Node-compatible; Node remains the mainstream production choice. Node has closed some gaps: it runs `.ts` files by type stripping (Q216) and has an opt-in, now stable **permission model** (`node --permission --allow-fs-read=./data app.js`). → *More:* §15.12

---

12. TypeScript basics

Q207. ⋆ What is TypeScript, and why use it? TypeScript is JavaScript plus a static type system: you annotate (or let it infer) types, and the compiler `tsc` reports type errors **before** the code runs; the types are then erased and plain JavaScript runs. Benefits: catches typos and wrong-shape bugs early, much better editor autocomplete and refactoring, types document APIs. Costs: a build/check step and learning curve. It does not check data at run time (validate external input with a schema library). → *More:* §16.8

Q208. ⋆ What is the difference between `interface` and `type`? Both describe object shapes and are mostly interchangeable. `interface` can be **reopened** (declaration merging — two declarations with the same name combine, used to augment library types) and extended with `extends`. `type` aliases can name **any** type — unions (`'a' | 'b'`), tuples, primitives, mapped and conditional types — and combine with `&`. Common

rule: `interface` for public object shapes, `type` for everything else; be consistent. → *More:* §16.10 Everyday types

Q209. ⋆ `any` vs `unknown` vs `never`? `any` turns type checking off for that value — anything goes, errors spread silently. `unknown` means "could be anything, so you must check before using it" — the safe type for external input (`JSON.parse`, `catch` errors). `never` is the type with no values: a function that always throws or never returns, or a branch that should be impossible (used for exhaustive `switch` checks). → *More:* §16.10 · §16.11 Narrowing

Q210. What is type narrowing? What are type guards and discriminated unions? Narrowing is TypeScript refining a union type inside a branch after a check: `typeof x === 'string'`, `x instanceof Date`, `'key' in x`, `Array.isArray(x)`, truthiness, equality. A **type guard** is a function returning `x is Type`, so its callers get narrowing. A **discriminated union** is a union of object types sharing a literal field (`kind: 'circle' | 'square'`); switching on that field narrows to the right member, and a `never` default catches missing cases. → *More:* §16.11

Q211. ⋆ What are generics? Type parameters that let one function or type work with many types while keeping them linked: `function first<T>(arr: T[]): T | undefined` returns the same type the array holds. Constraints restrict them: `<T extends { id: string }>`. Used everywhere in library types: `Array<T>`, `Promise<T>`, `Map<K, V>`. → *More:* §16.12 Generics and utility types

Q212. Name some utility types. `Partial<T>` (all optional), `Required<T>`, `Readonly<T>`, `Pick<T, 'a' | 'b'>`, `Omit<T, 'password'>`, `Record<K, V>` (object with keys K), `ReturnType<typeof fn>`, `Parameters<typeof fn>`, `Awaited<P>` (unwrap a promise), `NonNullable<T>`, `Exclude` / `Extract` (filter union members). They are built from mapped and conditional types. → *More:* §16.12

Q213. What do `keyof` and `typeof` mean in TypeScript types? In type position, `typeof value` gives the type of a JavaScript value (`type Config = typeof defaultConfig`). `keyof T` gives the union of T's keys (`keyof { a: 1; b: 2 }` is `'a' | 'b'`). Together: `function get<T, K extends keyof T>(obj: T, key: K): T[K]` — type-safe property access, where `T[K]` is an indexed access type. → *More:* §16.12

Q214. Enums vs union of string literals vs `as const`? What does `satisfies` do? A string-literal union (`type Status = 'idle' | 'loading'`) costs nothing at run time and is usually preferred. `enum` generates a JavaScript object and is not supported by Node's type stripping. An `as const` object (`const Status = { Idle: 'idle' } as const`) gives a runtime object plus literal types — the common enum replacement. `value satisfies Type` checks that a value matches a type **without** widening it, so you keep the precise inferred type (e.g. which keys exist). → *More:* §16.13

Q215. What is structural typing? TypeScript compares types by **shape**, not by name: any object with the required properties is compatible, even if declared separately ("duck typing" checked at compile time). Object literals passed directly get an extra check for unknown properties (excess property checking). → *More:* §16.10

Q216. Can Node 24 run TypeScript directly? Yes: `node app.ts` works without flags by **stripping** type annotations (stable in Node 24.12). It does no type checking, so still run `tsc --noEmit`; it supports only erasable syntax (no `enum`, namespaces with code, or parameter properties — enable `erasableSyntaxOnly`), needs `import type` for type-only imports (enforce with `verbatimModuleSyntax`), requires real `.ts` extensions in imports, ignores `tsconfig.json`, and won't strip files in `node_modules`. → *More:* §16.9

Q217. What are the pros and cons of languages that compile to JavaScript? Pros: types (TypeScript) or other features, earlier error detection, better tooling. Cons: a build step, source maps needed for debugging, possible mismatch between the language and the JavaScript it produces, an extra dependency, and team learning cost. TypeScript is the exception that became mainstream because it is "JavaScript plus types", not a different language. → *More:* §16.8

---

13. Output-prediction puzzles ("What will this print?")

Say the answer out loud **before** reading it. Each answer was checked by running the code in Node; files are ES modules (strict mode) unless the question says otherwise. Many more, with real output, are in §17.4 and §12.3.

Q218. ⋆ Timers, promises and `queueMicrotask`.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
queueMicrotask(() => console.log('4'));
console.log('5');
```

`1 5 3 4 2`. Synchronous code first (1, 5); then the microtask queue in order of queuing (3, 4); then the timer task (2). → *More:* §17.4.5 Q35

Q219. ⋆ Nested tasks and microtasks.

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve()
  .then(() => { console.log('C'); setTimeout(() => console.log('D'), 0); })
  .then(() => console.log('E'));
(async () => { console.log('F'); await null; console.log('G'); })();
console.log('H');
```

`A F H C G E B D`. The async function runs synchronously up to its first `await` (F). Microtasks queued during sync code: C, then G. Running C queues E (the next `then`) and timer D. Microtasks drain completely (C, G, E) before any timer; timers then run in the order created (B, D).

Q220. ⋆ `async` / `await` ordering.

```
async function a() { console.log('a1'); await b(); console.log('a2'); }
async function b() { console.log('b'); }
console.log('start');
a();
Promise.resolve().then(() => console.log('p'));
console.log('end');
```

`start a1 b end a2 p`. Calling `b()` runs its body synchronously. `await` of an already-fulfilled promise queues the continuation (a2) as a microtask — before `p`, which is queued later. → *More:* §17.4.5 Q36

Q221. ⋆ The promise executor.

```
const p = new Promise((resolve) => { console.log('A'); resolve('B'); console.log('C'); });
p.then(console.log);
console.log('D');
```

`A C D B`. The executor runs synchronously, and `resolve` does not stop it (C still prints). `then` callbacks are always asynchronous.

Q222. ⋆ Errors flowing through a chain.

```
Promise.resolve(1)
  .then((x) => { throw new Error('boom ' + x); })
  .then(() => console.log('skipped'))
  .catch((e) => { console.log(e.message); return 2; })
  .then((x) => console.log('after', x))
  .finally(() => console.log('finally'));
```

`boom 1`, `after 2`, `finally`. A throw skips `then` handlers until a `catch`; a `catch` that returns a value recovers the chain; `finally` receives no value and passes the previous result through.

Q223. What does an `async` function return?

```
async function f() { return 1; }
console.log(f());
```

`Promise { 1 }` (in Node). An `async` function always returns a promise; use `await f()` or `f().then(…)` to get `1`.

Q224. ⋆ `var` hoisting inside a function.

```
var x = 1;
function f() { console.log(x); var x = 2; }
f();
```

`undefined`. The inner `var x` is hoisted to the top of `f` and shadows the outer `x`, so at the `console.log` the local `x` exists but is still `undefined`. With `let x = 2` it would throw `ReferenceError` (TDZ). → *More:* §17.4.2 Q16

Q225. Function vs variable hoisting (a classic script).

```
console.log(typeof foo);
var foo = 1;
function foo() {}
console.log(typeof foo);
```

`function`, then `number`. Both declarations are hoisted; the function declaration wins at the start. Then the assignment `foo = 1` runs and replaces it.

Q226. ⋆ `this` in methods, arrows and nested functions.

```
const obj = {
  name: 'obj',
  regular() { return this.name; },
  arrow: () => this?.name,
  nested() { return [function () { return this?.name; }(), (() => this.name)()]; },
};
console.log(obj.regular(), obj.arrow(), obj.nested());
```

`obj undefined [ undefined, 'obj' ]`. `regular` is called on `obj`. The arrow takes `this` from module top level, which is `undefined`. Inside `nested`, the inner regular function is a plain call (`this` is `undefined` in strict code), while the inner arrow inherits `nested`'s `this`. → *More:* §17.4.3 Q25

Q227. A detached method (a CommonJS file, sloppy mode).

```
const user = { name: 'Ann', greet() { return 'Hi ' + this?.name; } };
const g = user.greet;
console.log(user.greet(), g());
```

`Hi Ann Hi undefined`. `g()` is a plain call: in sloppy mode `this` is the global object, which has no `name`. In strict mode `this` would be `undefined` (`this?.name` avoids the crash here; `this.name` would throw).

Q228. Class field arrow vs prototype method.

```
class Counter { count = 0; inc() { this.count++; } incArrow = () => { this.count++; }; }
const c = new Counter();
const { inc, incArrow } = c;
incArrow();
try { inc(); } catch (e) { console.log(e.constructor.name); }
console.log(c.count);
```

`TypeError`, then `1`. The arrow field is created per instance and captures it; the prototype method loses `this` when extracted (class bodies are strict, so `this` is `undefined`).

Q229. Closures keep separate state.

```
function make() { let c = 0; return { inc: () => ++c, get: () => c }; }
const a = make(), b = make();
a.inc(); a.inc(); b.inc();
console.log(a.get(), b.get());
```

`2 1`. Each call to `make` creates a new `c`; the two functions returned by one call share it.

Q230. ⋆ Coercion with `+` and `-`.

```
console.log("5" + 3 - 3, "5" - 3 + 3);
console.log([1, 2, 3] + [4], null + 1, undefined + 1);
console.log(1 < 2 < 3, 3 > 2 > 1);
```

`50 5` — `"5" + 3` is `"53"`, minus 3 is `50`; `"5" - 3` is `2`, plus 3 is `5`. `1,2,34 1 NaN` — arrays become strings; `null` becomes `0`, `undefined` becomes `NaN`. `true false` — `1 < 2` is `true`, `true < 3` is `1 < 3`; `3 > 2` is `true`, `true > 1` is `1 > 1`. → *More:* §17.4.1 Q1

Q231. `Number()` vs `parseInt()` on odd inputs.

```
console.log(Number(''), Number(' 12 '), Number('12px'), Number(null), Number(undefined));
console.log(Number([]), Number([5]), Number([1, 2]), parseInt('12px'), parseInt(0.0000005));
```

`0 12 NaN 0 NaN`, then `0 5 NaN 12 5`. Arrays convert via their string form (`''`, `'5'`, `'1,2'`). `parseInt(0.0000005)` is `5` because the number is first turned into the string `'5e-7'`, and parsing stops at `e`.

Q232. An object that equals three different numbers.

```
let count = 0;
const obj = { valueOf() { return ++count; } };
console.log(obj == 1, obj == 2, obj == 3);
```

`true true true`. `==` with a number converts the object with `valueOf` each time, and this one returns a new number on every call. (`===` would give `false` three times.)

Q233. ⋆ Default `sort`.

```
console.log([3, 20, 100].sort(), ['b', 'a', 'C'].sort());
```

`[ 100, 20, 3 ] [ 'C', 'a', 'b' ]`. Elements are compared as strings by UTF-16 code unit: `'1' < '2' < '3'`, and uppercase letters come before lowercase. Use `(a, b) => a - b` and `localeCompare`.

Q234. `map` with a block body.

```
console.log([1, 2, 3].map((n) => { n * 2; }));
```

`[ undefined, undefined, undefined ]`. Braces make a block body, which needs an explicit `return`. Write `(n) => n * 2`.

Q235. Holes and `length`.

```
const arr = [1, 2, 3];
arr[10] = 11;
console.log(arr.length, arr.filter(() => true).length);
```

`11 4`. Writing index 10 extends `length` to 11 and leaves holes at 3–9; `filter` skips holes.

Q236. `includes` vs `indexOf` with `NaN`, and `indexOf` with types.

```
console.log([NaN].includes(NaN), [NaN].indexOf(NaN), [1, 2, 3].indexOf('2'));
```

`true -1 -1`. `includes` uses SameValueZero (treats `NaN` as equal to itself); `indexOf` uses `===`, so it finds neither `NaN` nor the string `'2'`.

Q237. Constructors that return values.

```
function Foo() { this.a = 1; return { b: 2 }; }
function Bar() { this.a = 1; return 5; }
console.log(new Foo().a, new Foo().b, new Bar().a);
```

`undefined 2 1`. Returning an object from a constructor replaces the new object; returning a primitive is ignored. → *More:* §17.4.3 Q28

Q238. `return` inside `finally`.

```
function f() { try { return 'try'; } finally { console.log('cleanup'); } }
function g() { try { return 'try'; } finally { return 'finally'; } }
console.log(f()); console.log(g());
```

`cleanup`, `try`, `finally`. `finally` runs before the function actually returns; a `return` in `finally` overrides the one in `try`.

Q239. Generators and `next(value)`.

```
function* gen() { const x = yield 1; console.log('got', x); yield x * 2; }
const it = gen();
console.log(it.next().value); console.log(it.next(5).value); console.log(it.next().done);
```

`1`, `got 5`, `10`, `true`. The first `next()` runs to the first `yield`. The value passed to the second `next(5)` becomes the result of that `yield` expression.

Q240. Object key order.

```
console.log(Object.keys({ b: 1, 2: 1, a: 1, 1: 1 }));
```

`[ '1', '2', 'b', 'a' ]`. Integer-like keys come first in ascending numeric order, then string keys in insertion order (and keys are always strings).

Q241. Strings are immutable.

```
const s = 'abc';
s[0] = 'z';
```

In an ES module (strict mode) this throws `TypeError: Cannot assign to read only property '0' of string 'abc'`. In sloppy mode the assignment is silently ignored and `s` stays `'abc'`.

Q242. Mixing `??` with `||`.

```
console.log(0 || null ?? 'x');
```

`SyntaxError`. The language forbids mixing `??` with `||` or `&&` without parentheses, because the intended precedence is unclear. Write `(0 || null) ?? 'x'` (gives `'x'`).

Q243. ⋆ Loop closures with `let`.

```
const fns = [];
for (let i = 0; i < 3; i++) fns.push(() => i);
console.log(fns.map((fn) => fn()));
```

`[ 0, 1, 2 ]`. `let` in a `for` header creates a new binding for each iteration; with `var` it prints `[ 3, 3, 3 ]`. → *More:* §17.4.2 Q18

Q244. Array-to-primitive comparisons.

```
console.log(String([null]), String([undefined, null]), [] == '', [0] == false);
```

An empty string, `,`, `true`, `true`. `join` turns `null` / `undefined` into empty strings; `[]` becomes `''`; `[0]` becomes `'0'`, then `0`, and `false` becomes `0`.

---

14. Coding tasks: implement-from-scratch classics

Every snippet below was run against tests. In the interview, first state the contract (inputs, outputs, edge cases), write the simple version, then mention extensions (cancel, `this`, errors, leading edge). Algorithm and data-structure problems (two pointers, sliding window, BFS, DP, heaps…) are in §17.2 Coding patterns.

Q245. ⋆ Implement `debounce`.

```
function debounce(fn, wait) {
  let timer;
  const debounced = function (...args) {
    clearTimeout(timer);                                  // every call restarts the wait
    timer = setTimeout(() => fn.apply(this, args), wait);   // keep the caller's this and latest args
  };
  debounced.cancel = () => clearTimeout(timer);
  return debounced;
}
```

Follow-ups: a `leading` option (run on the first call, then ignore until quiet), `flush()` (run the pending call now), and returning a promise of the result. → *More:* §17.3 Debounce

Q246. ⋆ Implement `throttle`.

```
function throttle(fn, wait) {
  let last = 0, timer = null, lastArgs;
  return function (...args) {
    const now = Date.now();
    lastArgs = args;
    if (now - last >= wait) { last = now; fn.apply(this, args); }          // leading call
    else if (!timer) timer = setTimeout(() => {                             // one trailing call
      last = Date.now(); timer = null; fn.apply(this, lastArgs);
    }, wait - (now - last));
  };
}
```

Runs at most once per `wait`, with a trailing call so the last value is not lost. → *More:* §17.3 Throttle

Q247. ⋆ Implement `Promise.all`.

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], results = new Array(items.length);
    let left = items.length;
    if (left === 0) return resolve(results);                // empty input resolves at once
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {                    // also accepts plain values
        results[i] = v;                                      // keep input order
        if (--left === 0) resolve(results);
      }, reject);                                            // first rejection wins
    });
  });
}
```

→ *More:* §17.3 Promise polyfills

Q248. Implement `Promise.allSettled`.

```
const promiseAllSettled = (iterable) => promiseAll([...iterable].map((p) =>
  Promise.resolve(p).then((value) => ({ status: 'fulfilled', value }),
                          (reason) => ({ status: 'rejected', reason }))));
```

Each promise is turned into one that always fulfils with a status object, so `all` never rejects. → *More:* §17.3

Q249. ⋆ Implement `Promise.race` and `Promise.any`.

```
const promiseRace = (iterable) => new Promise((resolve, reject) => {
  for (const p of iterable) Promise.resolve(p).then(resolve, reject);     // first to settle wins
});
function promiseAny(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let left = items.length;
    if (left === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((p, i) => Promise.resolve(p).then(resolve, (e) => {
      errors[i] = e;
      if (--left === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

Extra calls to `resolve` / `reject` after the first are ignored, which is what makes `race` this short. `race([])` stays pending forever. → *More:* §17.3

Q250. ⋆ Implement `Function.prototype.bind`.

```
Function.prototype.myBind = function (ctx, ...preset) {
  const fn = this;
  return function bound(...later) {
    if (new.target) return new fn(...preset, ...later);   // `new` ignores the bound this
    return fn.apply(ctx, [...preset, ...later]);         // partial application
  };
};
```

→ *More:* §17.3 A `bind` polyfill

Q251. Implement `call` and `apply` without using them.

```
Function.prototype.myCall = function (ctx, ...args) {
  ctx = ctx == null ? globalThis : Object(ctx);    // box primitives
  const key = Symbol('fn');                        // unique key: no clash with ctx's own props
  ctx[key] = this;                                 // call it as a method so this === ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
Function.prototype.myApply = function (ctx, args = []) { return this.myCall(ctx, ...args); };
```

This mimics sloppy-mode behaviour (`null` becomes the global object, primitives are boxed); the real `call` passes `ctx` unchanged to a strict-mode function. → *More:* §17.3

Q252. ⋆ Implement a deep clone (with circular references).

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;    // primitives (and functions) as-is
  if (seen.has(value)) return seen.get(value);                        // cycle: reuse the copy
  if (value instanceof Date) return new Date(value);
  if (value instanceof Map) { const m = new Map(); seen.set(value, m);
    value.forEach((v, k) => m.set(deepClone(k, seen), deepClone(v, seen))); return m; }
  if (value instanceof Set) { const s = new Set(); seen.set(value, s);
    value.forEach((v) => s.add(deepClone(v, seen))); return s; }
  const copy = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);
  for (const key of Reflect.ownKeys(value)) copy[key] = deepClone(value[key], seen);  // incl. symbols
  return copy;
}
```

Mention that `structuredClone` is the built-in answer, and what it cannot copy (functions, DOM nodes, prototypes). → *More:* §17.3 Deep clone

Q253. ⋆ Implement deep equality.

```
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Array.isArray(a) !== Array.isArray(b)) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k]));
}
```

Extensions: `Date` / `Map` / `Set` support, and a `seen` map for cycles. → *More:* §17.3 Deep equal

Q254. ⋆ Flatten a nested array (with and without recursion).

```
const flatten = (arr, depth = Infinity) => depth < 1 ? arr.slice()
  : arr.reduce((acc, x) => (Array.isArray(x) ? acc.push(...flatten(x, depth - 1)) : acc.push(x), acc), []);

function flattenIterative(arr) {                    // no recursion: explicit stack
  const stack = [...arr], out = [];
  while (stack.length) { const x = stack.pop(); Array.isArray(x) ? stack.push(...x) : out.push(x); }
  return out.reverse();
}
```

The built-in is `arr.flat(depth)` (`flat(Infinity)` for all levels). Flattening an object to dotted keys (`{ 'a.b': 1 }`) is the same recursion over `Object.entries`. → *More:* §17.3 Flatten

Q255. ⋆ Implement `curry`.

```
function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length               // enough arguments: call the original
      ? fn.apply(this, args)
      : (...more) => curried.apply(this, [...args, ...more]);   // otherwise collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); add3(1, 2)(3); add3(1)(2, 3);       // all 6
```

→ *More:* §17.3 Curry

Q256. ⋆ Implement `sum(1)(2)(3)()` (infinite currying).

```
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
sum(1)(2)(3)();   // 6
```

Variant without the final `()`: return a function with a custom `valueOf` (`f.valueOf = () => total`) so `+sum(1)(2)(3)` or `sum(1)(2) == 3` works.

Q257. ⋆ Implement `memoize`.

```
function memoize(fn, keyFn = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyFn(...args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));   // has(): caches undefined results too
    return cache.get(key);
  };
}
```

Discuss the key (objects as arguments → `WeakMap` or a key function), memory growth (LRU limit or TTL), and memoising async functions (cache the promise and delete it on rejection). → *More:* §17.3 Memoize

Q258. ⋆ Implement an `EventEmitter` (`on`, `off`, `once`, `emit`).

```
class EventEmitter {
  #events = new Map();                                  // name -> array of listeners
  on(name, fn) {
    if (!this.#events.has(name)) this.#events.set(name, []);
    this.#events.get(name).push(fn);
    return () => this.off(name, fn);                    // handy unsubscribe function
  }
  off(name, fn) {
    const list = this.#events.get(name) ?? [];
    this.#events.set(name, list.filter((f) => f !== fn && f.original !== fn));
    return this;
  }
  once(name, fn) {
    const wrapper = (...args) => { this.off(name, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                              // so off(name, fn) also removes it
    return this.on(name, wrapper);
  }
  emit(name, ...args) {
    const list = this.#events.get(name);
    if (!list?.length) return false;
    [...list].forEach((f) => f.apply(this, args));      // copy: listeners may unsubscribe
    return true;
  }
}
```

→ *More:* §17.3 EventEmitter · §15.8

Q259. ⋆ Implement an LRU cache with O(1) `get` and `put`.

```
class LRUCache {
  #map = new Map();                          // Map remembers insertion order
  constructor(capacity) { this.capacity = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);          // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key); this.#map.set(key, value);
    if (this.#map.size > this.capacity) this.#map.delete(this.#map.keys().next().value); // evict oldest
  }
}
```

In other languages (or if asked to avoid `Map` ordering) use a hash map plus a doubly linked list. → *More:* §17.3 LRU cache

Q260. ⋆ Write polyfills for `map`, `filter` and `reduce`.

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = new Array(this.length);
  for (let i = 0; i < this.length; i++) if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  return out;
};
Array.prototype.myFilter = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) if (i in this && cb.call(thisArg, this[i], i, this)) out.push(this[i]);
  return out;
};
Array.prototype.myReduce = function (cb, ...init) {
  let i = 0, acc;
  if (init.length) acc = init[0];
  else { while (i < this.length && !(i in this)) i++;
    if (i >= this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[i++]; }
  for (; i < this.length; i++) if (i in this) acc = cb(acc, this[i], i, this);
  return acc;
};
```

Points interviewers check: skipping holes (`i in this`), passing `(value, index, array)`, `thisArg`, and `reduce` without an initial value. (Use regular functions, not arrows, so `this` is the array.)

Q261. Implement `once`.

```
function once(fn) {
  let done = false, result;
  return function (...args) {
    if (!done) { done = true; result = fn.apply(this, args); }
    return result;                      // later calls return the first result
  };
}
```

(The LeetCode version returns `undefined` on later calls — read the spec.)

Q262. Implement `sleep` and `promisify`.

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const promisify = (fn) => (...args) => new Promise((resolve, reject) =>
  fn(...args, (err, value) => (err ? reject(err) : resolve(value))));   // Node error-first callback
```

Node ships `util.promisify` and `node:timers/promises` (`await setTimeout(100)`).

Q263. ⋆ Implement retry with exponential backoff.

```
async function retry(fn, { retries = 3, delay = 100 } = {}) {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(attempt); }
    catch (err) {
      if (attempt >= retries) throw err;              // out of attempts: give up
      await sleep(delay * 2 ** attempt);              // 100, 200, 400 ms…
    }
  }
}
```

Mention adding random **jitter** so many clients don't retry in lockstep, retrying only transient errors (timeouts, 503), and an `AbortSignal`.

Q264. ⋆ Run async tasks with a concurrency limit (a promise pool).

```
async function runWithLimit(tasks, limit) {          // tasks: functions returning promises
  const results = new Array(tasks.length);
  let next = 0;
  async function worker() {
    while (next < tasks.length) { const i = next++; results[i] = await tasks[i](); }
  }
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

`limit` workers each pull the next task as soon as they finish one; `next++` is safe because JavaScript runs one piece of code at a time. → *More:* §12.13 Real-world async patterns

Q265. Implement the `new` operator and `Object.create`.

```
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);                  // 1. link the prototype
  const result = Ctor.apply(obj, args);                       // 2. run the constructor
  const isObject = result !== null && (typeof result === 'object' || typeof result === 'function');
  return isObject ? result : obj;                             // 3. object return wins
}
function objectCreate(proto) { function F() {} F.prototype = proto; return new F(); }
```

→ *More:* §9.4

Q266. Implement `instanceof`.

```
function myInstanceOf(obj, Ctor) {
  if (obj === null || (typeof obj !== 'object' && typeof obj !== 'function')) return false;
  for (let p = Object.getPrototypeOf(obj); p; p = Object.getPrototypeOf(p)) {
    if (p === Ctor.prototype) return true;
  }
  return false;
}
```

→ *More:* §9.7

Q267. Implement lodash-style `get(obj, 'a.b[0].c', fallback)`.

```
function get(obj, path, fallback) {
  const keys = Array.isArray(path) ? path : path.replace(/\[(\w+)\]/g, '.$1').split('.').filter(Boolean);
  let cur = obj;
  for (const k of keys) { if (cur == null) return fallback; cur = cur[k]; }
  return cur === undefined ? fallback : cur;
}
```

For a fixed path, optional chaining does this: `obj?.a?.b?.[0]?.c ?? fallback`.

Q268. Implement `setInterval` using `setTimeout`. Why would you?

```
function mySetInterval(fn, ms) {
  let id, stopped = false;
  const tick = () => { id = setTimeout(() => { fn(); if (!stopped) tick(); }, ms); };  // schedule the next run
after this one
  tick();
  return () => { stopped = true; clearTimeout(id); };   // the flag also works when fn() itself cancels
}
```

Without the `stopped` flag, calling the cancel function from inside `fn` would not work: `tick()` would schedule a new timer right after it. Chained timeouts guarantee a gap between runs; `setInterval` fires on schedule even if the previous (async) run has not finished, so calls can pile up.

Q269. Implement a basic `Promise` class.

```
class MyPromise {
  #state = 'pending'; #value; #handlers = [];
  constructor(executor) {
    const settle = (state, value) => {
      if (this.#state !== 'pending') return;                    // settle only once
      this.#state = state; this.#value = value;
      this.#handlers.forEach((h) => h());
    };
    try { executor((v) => settle('fulfilled', v), (e) => settle('rejected', e)); }
    catch (e) { settle('rejected', e); }
  }
  then(onFul, onRej) {
    return new MyPromise((resolve, reject) => {
      const run = () => queueMicrotask(() => {                  // callbacks are always async
        const cb = this.#state === 'fulfilled' ? onFul : onRej;
        if (typeof cb !== 'function') return (this.#state === 'fulfilled' ? resolve : reject)(this.#value);
        try { resolve(cb(this.#value)); } catch (e) { reject(e); }
      });
      this.#state === 'pending' ? this.#handlers.push(run) : run();
    });
  }
  catch(onRej) { return this.then(undefined, onRej); }
}
```

This shows the core ideas: three states, settle once, async callbacks, `then` returns a new promise. A full Promises/A+ version also **adopts** a returned promise or thenable (waits for it instead of fulfilling with it). → *More:* §12.7

Q270. Small warm-ups: FizzBuzz, `duplicate([1, 2, 3])`, unique, chunk, palindrome.

```
for (let i = 1; i <= 100; i++) console.log((i % 3 ? '' : 'fizz') + (i % 5 ? '' : 'buzz') || i);
const duplicate = (arr) => [...arr, ...arr];                    // [1,2,3,1,2,3]
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, n) => Array.from({ length: Math.ceil(arr.length / n) }, (_, i) => arr.slice(i * n, i * n +
n));
const isPalindrome = (s) => { const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); return t ===
[...t].reverse().join(''); };
```

→ *More:* §17.2 Coding patterns

---

15. LeetCode "30 Days of JavaScript"

LeetCode's **30 Days of JavaScript** study plan is the most common online practice set for JavaScript-specific interview tasks. Its sections are Closures, Basic Array Transformations, Function Transformations, Promises and Time, JSON, and Classes. The problems below follow the plan's order; extra JavaScript problems often practised alongside it come at the end. Read each LeetCode statement for exact edge cases; the answers here are compact versions that pass the stated examples.

Q271. Create Hello World Function. Return a function that always returns `"Hello World"`, whatever it is called with.

```
const createHelloWorld = () => (...args) => 'Hello World';
```

Tests that you know functions are values that can be returned (higher-order functions).

Q272. Counter. `createCounter(n)` returns a function that returns `n`, then `n + 1`, and so on.

```
const createCounter = (n) => () => n++;      // postfix: return the old value, then increment
```

The parameter `n` is captured by the closure and lives on between calls. → *More:* §4.7 Closures

Q273. To Be Or Not To Be. `expect(val)` returns `{ toBe, notToBe }`: `toBe(v)` returns `true` if `v === val`, otherwise throws `"Not Equal"`; `notToBe(v)` returns `true` if `v !== val`, otherwise throws `"Equal"`.

```
const expect = (val) => ({
  toBe: (v) => { if (v !== val) throw new Error('Not Equal'); return true; },
  notToBe: (v) => { if (v === val) throw new Error('Equal'); return true; },
});
```

This is the shape of Jest's `expect` — an object of closures.

Q274. Counter II. `createCounter(init)` returns `{ increment, decrement, reset }`, each returning the new value.

```
function createCounter(init) {
  let value = init;
  return { increment: () => ++value, decrement: () => --value, reset: () => (value = init) };
}
```

Q275. Apply Transform Over Each Element in Array. Implement `map(arr, fn)` without `Array.prototype.map`; `fn` receives `(arr[i], i)`.

```
function map(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) out.push(fn(arr[i], i)); return out;
}
```

Q276. Filter Elements from Array. Implement `filter(arr, fn)` keeping elements where `fn(arr[i], i)` is truthy, without `filter`.

```
function filter(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) if (fn(arr[i], i))
out.push(arr[i]); return out; }
```

Q277. Array Reduce Transformation. Implement `reduce(nums, fn, init)`; an empty array returns `init`.

```
function reduce(nums, fn, init) { let acc = init; for (const n of nums) acc = fn(acc, n); return acc; }
```

Q278. ⋆ Function Composition. `compose([f, g, h])` returns `x => f(g(h(x)))`; an empty list gives the identity function.

```
const compose = (functions) => (x) => functions.reduceRight((acc, fn) => fn(acc), x);
```

→ *More:* §17.3 Compose and pipe

Q279. Return Length of Arguments Passed.

```
const argumentsLength = (...args) => args.length;
```

Rest parameters collect any number of arguments into a real array.

Q280. ⋆ Allow One Function Call. `once(fn)` calls `fn` the first time and returns its result; later calls return `undefined`.

```
function once(fn) {
  let called = false;
  return (...args) => { if (called) return undefined; called = true; return fn(...args); };
}
```

Q281. Memoize. `memoize(fn)` returns a version that never calls `fn` twice with the same inputs (`sum(a, b)`, `fib(n)`, `factorial(n)` — note `sum(2, 3)` and `sum(3, 2)` are different inputs).

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = args.join(',');              // inputs are numbers here; use JSON.stringify for general data
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

→ *More:* Q257 above

Q282. Add Two Promises. Return a promise of the sum of two promises' numbers.

```
const addTwoPromises = async (p1, p2) => (await p1) + (await p2);
```

Both promises already run in parallel (they were created before the call); `Promise.all([p1, p2])` works too.

Q283. Sleep.

```
const sleep = (millis) => new Promise((resolve) => setTimeout(resolve, millis));
```

Q284. Timeout Cancellation. `cancellable(fn, args, t)` runs `fn(...args)` after `t` ms and returns `cancelFn`; calling `cancelFn` before then prevents the call.

```
function cancellable(fn, args, t) {
  const id = setTimeout(() => fn(...args), t);
  return () => clearTimeout(id);
}
```

Q285. Interval Cancellation. Call `fn(...args)` immediately and then every `t` ms until `cancelFn` is called.

```
function cancellable(fn, args, t) {
  fn(...args);
  const id = setInterval(() => fn(...args), t);
  return () => clearInterval(id);
}
```

Q286. ⋆ Promise Time Limit. `timeLimit(fn, t)` returns an async function that resolves like `fn`, or rejects with `"Time Limit Exceeded"` if `fn` takes longer than `t` ms.

```
const timeLimit = (fn, t) => (...args) => Promise.race([
  fn(...args),
  new Promise((_, reject) => setTimeout(() => reject('Time Limit Exceeded'), t)),
]);
```

In production, clear the losing timer and use `AbortSignal.timeout(t)` to actually stop the work.

Q287. ⋆ Cache With Time Limit. A class with `set(key, value, duration)` (returns `true` if an unexpired key already existed, and overwrites it), `get(key)` (value or `-1`) and `count()` (number of unexpired keys).

```
class TimeLimitedCache {
  #store = new Map();                                  // key -> { value, expires }
  set(key, value, duration) {
    const alive = this.get(key) !== -1;
    this.#store.set(key, { value, expires: Date.now() + duration });
    return alive;
  }
  get(key) { const e = this.#store.get(key); return e && e.expires > Date.now() ? e.value : -1; }
  count() { let n = 0; for (const e of this.#store.values()) if (e.expires > Date.now()) n++; return n; }
}
```

(Using `get` inside `set` assumes `-1` is never a stored value, which holds for this problem.) Alternative: a `setTimeout` per key that deletes it (clear the old timer on overwrite).

Q288. Debounce. Same as the classic: delay each call by `t` ms and cancel the previous pending call.

```
function debounce(fn, t) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), t); };
}
```

→ *More:* §17.3 Debounce

Q289. ⋆ Execute Asynchronous Functions in Parallel. Given an array of functions that return promises, implement `promiseAll(functions)` without `Promise.all`.

```
function promiseAll(functions) {
  return new Promise((resolve, reject) => {
    const results = new Array(functions.length);
    let left = functions.length;
    if (left === 0) return resolve(results);
    functions.forEach((fn, i) => fn().then((v) => {
      results[i] = v;
      if (--left === 0) resolve(results);
    }, reject));
  });
}
```

→ *More:* §17.3 Promise polyfills

Q290. Is Object Empty. Return `true` for `{}` or `[]`.

```
const isEmpty = (obj) => Object.keys(obj).length === 0;
```

A version that stops at the first key without building a key array is `for (const _ in obj) return false; return true;` — but it breaks if anyone has added enumerable properties to `Array.prototype` (as the "Array Prototype Last" solution below does with plain assignment).

Q291. Chunk Array. Split `arr` into sub-arrays of length `size` (the last may be shorter), without lodash's `_.chunk`.

```
function chunk(arr, size) {
  const out = [];
  for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
  return out;
}
```

Q292. Array Prototype Last. Add `arr.last()` returning the last element, or `-1` for an empty array.

```
Array.prototype.last = function () { return this.length ? this[this.length - 1] : -1; };
```

Use a regular function so `this` is the array. In real code use the built-in `arr.at(-1)`, and avoid extending built-ins (Q69). If you must, use `Object.defineProperty` so the method is non-enumerable.

Q293. ⋆ Group By. Add `arr.groupBy(fn)` returning an object whose keys are `fn(item)` and values are arrays of items.

```
Array.prototype.groupBy = function (fn) {
  const out = {};
  for (const item of this) (out[fn(item)] ??= []).push(item);
  return out;
};
```

The standard version is `Object.groupBy(arr, fn)` (ES2024). → *More:* §8.7

Q294. Sort By. Return `arr` sorted ascending by the number `fn(item)`.

```
const sortBy = (arr, fn) => arr.toSorted((a, b) => fn(a) - fn(b));
```

Q295. Join Two Arrays by ID. Merge two arrays of objects by `id`; for the same `id`, properties from `arr2` override `arr1`; return the result sorted by `id`.

```
function join(arr1, arr2) {
  const byId = new Map();
  for (const o of arr1) byId.set(o.id, { ...o });
  for (const o of arr2) byId.set(o.id, { ...byId.get(o.id), ...o });   // later spread wins
  return [...byId.values()].sort((a, b) => a.id - b.id);
}
```

Q296. ⋆ Flatten Deeply Nested Array. Implement `flat(arr, n)` (flatten up to depth `n`) without `Array.prototype.flat`.

```
function flat(arr, n) {
  if (n === 0) return arr.slice();
  const out = [];
  for (const x of arr) Array.isArray(x) ? out.push(...flat(x, n - 1)) : out.push(x);
  return out;
}
```

Q297. Compact Object. Remove all falsy values from an object or array, recursively (`compactObject([null, 0, 5, [0], [false, 16]])` → `[5, [], [16]]`).

```
function compactObject(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.filter(Boolean).map(compactObject);
  const out = {};
  for (const [k, v] of Object.entries(obj)) if (v) out[k] = compactObject(v);
  return out;
}
```

Q298. ⋆ Event Emitter. `subscribe(eventName, callback)` returns `{ unsubscribe() }`; `emit(eventName, args = [])` calls the callbacks in subscription order and returns an array of their results.

```
class EventEmitter {
  #events = new Map();                                  // name -> Set of entries
  subscribe(eventName, callback) {
    if (!this.#events.has(eventName)) this.#events.set(eventName, new Set());
    const subs = this.#events.get(eventName), entry = { callback };  // wrapper: same callback twice is fine
    subs.add(entry);
    return { unsubscribe: () => { subs.delete(entry); } };
  }
  emit(eventName, args = []) {
    return [...(this.#events.get(eventName) ?? [])].map(({ callback }) => callback(...args));
  }
}
```

→ *More:* Q258 above

Q299. Array Wrapper. Class `ArrayWrapper(nums)` where `obj1 + obj2` gives the sum of all elements and `String(obj)` gives `"[1,2,3]"`.

```
class ArrayWrapper {
  constructor(nums) { this.nums = nums; }
  valueOf() { return this.nums.reduce((a, b) => a + b, 0); }   // used by +
  toString() { return `[${this.nums.join(',')}]`; }            // used by String()
}
```

→ *More:* Q17 above (object-to-primitive conversion)

Q300. ⋆ Calculator with Method Chaining. `new Calculator(10).add(5).subtract(7).getResult()` → `8`; `divide(0)` throws `"Division by zero is not allowed"`.

```
class Calculator {
  constructor(value) { this.value = value; }
  add(v) { this.value += v; return this; }              // return this -> chainable
  subtract(v) { this.value -= v; return this; }
  multiply(v) { this.value *= v; return this; }
  divide(v) { if (v === 0) throw new Error('Division by zero is not allowed'); this.value /= v; return this; }
  power(v) { this.value **= v; return this; }
  getResult() { return this.value; }
}
```

Q301. Check if Object Instance of Class. Return whether a value is an instance of a class, including primitives (`5` is an instance of `Number`) and without throwing for odd inputs.

```
function checkIfInstanceOf(obj, classFunction) {
  if (obj == null || typeof classFunction !== 'function') return false;
  for (let p = Object.getPrototypeOf(obj); p !== null; p = Object.getPrototypeOf(p)) {
    if (p === classFunction.prototype) return true;      // getPrototypeOf boxes primitives
  }
  return false;
}
```

Unlike `instanceof`, this says `5` is a `Number`, because `Object.getPrototypeOf(5)` is `Number.prototype`.

Q302. Generate Fibonacci Sequence and Nested Array Generator. Write a generator yielding Fibonacci numbers forever, and one yielding the integers of a nested array in order.

```
function* fibGenerator() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
function* inorderTraversal(arr) { for (const x of arr) Array.isArray(x) ? yield* inorderTraversal(x) : yield x; }
```

`yield*` delegates to another generator. Infinite generators are safe because values are produced only on demand. → *More:* §8.5 Generators

Q303. Throttle, Curry, JSON Deep Equal, Promise Pool, Call Function with Custom Context, Undefined to Null. These JavaScript problems are also commonly practised with the plan; each matches a classic above:

- **Throttle** and **Curry** → the throttle and curry answers in group 14.
- **JSON Deep Equal** → the deep-equality answer (JSON values only, so no `Date` / `Map` handling is needed).

- **Promise Pool** (`promisePool(functions, n)`) → the concurrency-limit answer, without collecting results.
- **Call Function with Custom Context** (`fn.callPolyfill(context, ...args)`) → the `call` polyfill.
- **Undefined to Null** → a recursive walk, or `JSON.parse(JSON.stringify(obj, (k, v) => (v === undefined ? null : v)))`.

Sources of the question lists

Question wording and frequency were taken from these lists and posts (titles only; all answers are this book's own):

- LeetCode — *30 Days of JavaScript* study plan (leetcode.com/studyplan/30-days-of-javascript); problem lists as mirrored in public solution repositories (e.g. github.com/Rizwan-Damani/leetcode-30-days-js, github.com/mahmoudabozied4/30-Days-of-JavaScript-LeetCode).
- sudheerj, *javascript-interview-questions* (github.com/sudheerj/javascript-interview-questions) — the 500+ question list.
- h5bp, *Front-end Developer Interview Questions* — JavaScript questions and coding questions (github.com/h5bp/Front-end-Developer-Interview-Questions).
- GreatFrontEnd — *Front End Interview Playbook: JavaScript coding* and *10 Must-Know JavaScript Coding Interview Questions* (greatfrontend.com).
- NamasteDev — "JavaScript interview questions by company" (namastedev.com).
- Facts were checked against MDN Web Docs, the ECMAScript specification and TC39 finished-proposals list, the Node.js documentation, and OWASP cheat sheets, as for the rest of this book.